سیستم‌های توصیه‌گر

جزوه‌ی کامل هفته‌های ۱ تا ۳ — از Baseline تا LightGCN، MF، Collaborative Filtering و Content-based

مفهوم، ریاضی، مثال‌های حل‌شده، و پاسخ سوال‌های استاد

بر اساس نوت‌بوک‌های درس: baselines · knn_cf · gnn · content_based

دیتاست درس: MovieLens (ml-latest-small)

فهرست مطالب

راهنمای استفاده از این جزوه

فصل ۰ — مسئله‌ی توصیه، از پایه

۰.۱ سیستم توصیه‌گر چه مسئله‌ای را حل می‌کند؟

۰.۲ جدول کوچکی که در کل جزوه استفاده می‌کنیم

۰.۳ Sparsity: چرا مسئله سخت است

۰.۴ دو نوع داده: Explicit و Implicit

۰.۵ دو وظیفه‌ی متفاوت: Rating Prediction و Top-N

۰.۶ نقشه‌ی سه هفته: هر روش، جواب ضعف روش قبلی

فصل ۱ — Baselines (هفته‌ی ۱)

۱.۱ مسئله: روز اول، بدون هیچ مدلی

۱.۲ Baseline های پیش‌بینی امتیاز

۱.۳ اندازه‌گیری: MAE و RMSE

۱.۴ Baseline های Top-N: صفحه‌ی اول را چه بگذاریم؟

۱.۵ اندازه‌گیری Top-N: Hit Ratio

۱.۶ الگوریتم و پیچیدگی

۱.۷ کد استاد و ایرادهایش

۱.۸ سوال‌های امتحان — فصل ۱

فصل ۲ — Collaborative Filtering مبتنی بر همسایگی (هفته‌ی ۲، بخش اول)

۲.۱ مسئله: «چون X را دیدی...»

۲.۲ برنامه‌ی کار

۲.۳ Similarity اول: Jaccard

۲.۴ Similarity دوم: Cosine

۲.۵ Similarity سوم: Adjusted cosine (mean-centered)

۲.۶ از similarity به پیش‌بینی

۲.۷ Top-N: ساختن ردیف «چون X را دیدی»

۲.۸ نگه داشتن k بهترین: لیست مرتب محدود و Heap

۲.۹ مسئله‌ی هزینه و الگوریتم آمازون

۲.۱۰ الگوریتم‌ها و پیچیدگی

۲.۱۱ کد استاد و ایرادهایش

۲.۱۲ سوال‌های امتحان — فصل ۲

فصل ۳ — Matrix Factorization، BPR و LightGCN (هفته‌ی ۲، بخش دوم)

۳.۱ مسئله: کاربران امتیاز نمی‌دهند، و کاتالوگ بزرگ است

۳.۲ معیارهای رتبه‌بندی: Precision، Recall و NDCG

۳.۳ Matrix Factorization: توصیف کاربر و فیلم با چند عدد

۳.۴ BPR: بردارها از داده‌ی implicit چطور یاد گرفته می‌شوند؟

۳.۵ LightGCN: اشتراک اطلاعات بین همسایه‌ها در گراف

۳.۶ کد استاد (`gnn.ipynb`) و ایرادهایش

۳.۷ سوال‌های امتحان — فصل ۳

فصل ۴ — Content-based Recommendation (هفته‌ی ۳)

۴.۱ مسئله: فیلم جدید، صفر امتیاز
۴.۲ نمایش کالا
۴.۳ شباهت ژانر با Jaccard
۴.۴ پیش‌بینی امتیاز: همان میانگین وزن‌دار
۴.۵ بردار و cosine: one-hot، TF-IDF و embedding
۴.۶ Top-N با پروفایل کاربر
۴.۷ ارزیابی منصفانه: split زمانی، Hit Ratio و NDCG
۴.۸ Content در برابر CF، و روش‌های ترکیبی (Hybrid)
۴.۹ الگوریتم‌ها و پیچیدگی
۴.۱۰ کد استاد (`content_based.ipynb`) و ایرادهایش
۴.۱۱ سوال‌های امتحان — فصل ۴

فصل ۵ — جمع‌بندی، فرمول‌نامه و تمرین‌های مرور

۵.۱ کل درس در یک جدول
۵.۲ فرمول‌نامه
۵.۳ عددهای مرجع
۵.۴ اشتباه‌های رایج در امتحان
۵.۵ تمرین‌های مرور (با جواب)

راهنمای استفاده از این جزوه

این جزوه همه‌ی مطالب چهار نوت‌بوک استاد را پوشش می‌دهد:

هفته	نوت‌بوک استاد	فصل این جزوه
هفته‌ی ۱	`baselines.ipynb`	فصل ۱ — Baselines
هفته‌ی ۲	`knn_cf.ipynb`	فصل ۲ — Collaborative Filtering مبتنی بر همسایگی
هفته‌ی ۲	`gnn.ipynb`	فصل ۳ — Matrix Factorization، BPR و LightGCN
هفته‌ی ۳	`content_based.ipynb`	فصل ۴ — Content-based Recommendation

ساختار هر فصل یکسان است:

1. **مسئله:** این روش جواب چه مشکلی است و کِی سراغش می‌رویم.
2. **ایده و ریاضی:** فرمول‌ها، با توضیح این‌که هر جزء فرمول چرا آن‌جاست.
3. **مثال‌های حل‌شده:** برای هر مفهوم حداقل دو مثال، قدم‌به‌قدم، طوری که خودت بتوانی روی کاغذ تکرارش کنی.
4. **الگوریتم و پیچیدگی زمانی:** شبه‌کد و هزینه‌ی محاسباتی.
5. **کد استاد:** چه کار می‌کند و چه ایرادهایی دارد (سوال «این کد چه مشکلی دارد؟» محتمل است).
6. **سوال‌های امتحان:** سوال‌های خود استاد با جواب، و شکل‌های دیگری که ممکن است بپرسد.

یک داستان ثابت در کل جزوه. فرض کن مهندس یادگیری ماشین یک سرویس استریم فیلم هستی. هر فصل جواب یک درخواست واقعی مدیر محصول است، و هر روش دقیقاً ضعف روش قبلی را برطرف می‌کند. برای همین مطالب پشت سر هم و مرتبط‌اند، نه جدا از هم.

نشانه‌ها:

> **کادر سبز = مثال حل‌شده.**

> **کادر آبی = «چرا؟»** — دلیل وجود یک فرمول یا یک قدم.

> **کادر نارنجی = سوال امتحانی** با جواب نمونه. جواب‌ها به انگلیسی هم آمده‌اند، چون امتحان انگلیسی است.

> **کادر قرمز = اشتباه رایج یا ایراد کد استاد.**

> **کادر بنفش = دید صنعتی** — این روش در شرکت‌های واقعی چطور استفاده می‌شود.

فصل ۰ — مسئله‌ی توصیه، از پایه

۰.۱ سیستم توصیه‌گر چه مسئله‌ای را حل می‌کند؟

یک سرویس مثل Netflix یا Amazon میلیون‌ها کالا دارد، ولی هر کاربر فقط چند ده تای آن‌ها را می‌بیند. سیستم توصیه‌گر باید از بین همه‌ی کالاها، **چند مورد را که این کاربر احتمالاً دوست دارد** انتخاب کند.

تنها ورودی اصلی ما یک جدول است: **ماتریس کاربر–کالا (user–item matrix)**. هر سطر یک کاربر، هر ستون یک کالا، و هر خانه یعنی آن کاربر با آن کالا چه کرده است.

۰.۲ جدول کوچکی که در کل جزوه استفاده می‌کنیم

چهار کاربر و پنج فیلم. عدد یعنی امتیاز (۱ تا ۵ ستاره) و `?` یعنی کاربر آن فیلم را امتیاز نداده است.

Titanic	Terminator	Matrix	Nemo	Toy Story	
?	?	1	4	5	**Ana**
?	4	5	?	1	**Ben**
3	2	?	5	4	**Cem**
1	5	4	2	?	**Dee**

جدول را مثل یک انسان بخوان:

- **Ana** انیمیشن‌ها را دوست دارد (۵ و ۴) و از Matrix بدش می‌آید (۱).
- **Ben** برعکس است: Toy Story را ۱ داده و فیلم‌های اکشن را ۵ و ۴.
- **Cem** شبیه Ana است و **Dee** شبیه Ben.

سوالی که در کل جزوه دنبالش می‌کنیم:

> **آیا باید فیلم Terminator را به Ana پیشنهاد بدهیم؟**

شهود تو می‌گوید **نه**: Ana از Matrix بدش آمده، و کسانی که Terminator را دوست دارند (Ben و Dee) همان کسانی‌اند که Matrix را دوست دارند. در هر فصل می‌بینیم کدام روش این را درست تشخیص می‌دهد و کدام نه، و **چرا**.

۰.۳ Sparsity: چرا مسئله سخت است

تعریف. Density یعنی سهم خانه‌های پرشده‌ی جدول، و **Sparsity** یعنی سهم خانه‌های خالی:

$$\text{density} = \frac{\text{number of ratings}}{\text{users} \times \text{items}}, \qquad \text{sparsity} = 1 - \text{density}$$

مثال ۱ (جدول کوچک). ۱۴ امتیاز داریم و جدول $4 \times 5 = 20$ خانه دارد.

$\text{density} = 14/20 = 0.70$ و $\text{sparsity} = 1 - 0.70 = 0.30$، یعنی ۳۰٪ خانه‌ها خالی‌اند.

مثال ۲ (دیتاست درس، MovieLens). ۱۰۰,۸۳۶ امتیاز، ۶۱۰ کاربر و ۹,۷۲۴ فیلم.

$$\text{density} = \frac{100836}{610 \times 9724} = \frac{100836}{5931640} = 0.017 \quad \text{و} \quad \text{sparsity} = 0.983.$$

یعنی **۹۸٫۳٪ جدول خالی است.** در داده‌های واقعی (مثلاً آمازون) این عدد از ۹۹٫۹۹٪ هم بیشتر است.

نکته‌ی امتحانی. استاد در کدش عدد `n_ratings/(n_users*n_items)` را با اسم *Sparsity* چاپ می‌کند. این عدد (۰٫۰۱۷) در واقع **density** است. اگر پرسیدند sparsity این دیتاست چقدر است، جواب حدود **۹۸٫۳٪** است.

دو واقعیت دیگر درباره‌ی داده‌ی واقعی که بارها به آن برمی‌گردیم:

- **Long tail (دُم بلند):** حدود نیمی از فیلم‌های MovieLens فقط ۱ یا ۲ امتیاز دارند. درباره‌ی این‌ها تقریباً هیچ نمی‌دانیم.
- **Popularity bias:** ۱۰٪ پرطرفدارترین فیلم‌ها حدود ۶۰٪ کل امتیازها را دارند.

۰.۴ دو نوع داده: Explicit و Implicit

	Explicit feedback	Implicit feedback
نمونه	ستاره، لایک/دیس‌لایک	کلیک، تماشا، خرید، check-in
خانه‌ی پر یعنی	«به این نمره‌ی ۴ می‌دهم»	«با این تعامل داشتم»
خانه‌ی خالی یعنی	«نمره نداده‌ام»	**نامعلوم**: یا دوستش ندارد، یا اصلاً ندیده
نمونه‌ی منفی	دارد (۱ و ۲ ستاره)	**ندارد**
حجم	کم	بسیار زیاد

در صنعت، داده‌ی implicit حالت عادی است، چون بیشتر کاربران هرگز ستاره نمی‌دهند. نوت‌بوک‌های `baselines` و `knn_cf` با داده‌ی explicit (MovieLens) کار می‌کنند و نوت‌بوک `gnn` با داده‌ی implicit (Gowalla)، یعنی «کاربر X در مکان Y check-in کرد».

۰.۵ دو وظیفه‌ی متفاوت: Rating Prediction و Top-N

همه‌ی مطالب این درس زیر یکی از این دو وظیفه است. سر هر سوال امتحان، اول از خودت بپرس سوال درباره‌ی کدام است، چون روش‌ها و معیارها فرق دارند.

	Rating Prediction	Top-N Recommendation
سوال	Ana به Terminator چه نمره‌ای می‌دهد؟	کدام N فیلم را در صفحه‌ی اول Ana نشان بدهیم؟
خروجی	یک عدد (مثلاً ۱٫۹)	یک لیست مرتب
نوع مسئله	رگرسیون (regression)	رتبه‌بندی (ranking)
معیار	MAE، RMSE (کمتر بهتر)	Hit Ratio، Precision@K، Recall@K، NDCG@K (بیشتر بهتر)
در صنعت	کم‌کاربرد	تقریباً همه‌ی سیستم‌های واقعی

چرا rating prediction در صنعت کم‌کاربرد است؟ کاربر نمره‌ی پیش‌بینی‌شده را نمی‌بیند؛ یک **لیست** می‌بیند. مدلی می‌تواند در حدس «۳٫۴ یا ۳٫۶» برای فیلم‌های متوسط خیلی دقیق باشد ولی در آوردن پنج فیلم عالی به بالای لیست بد باشد. این وظیفه با جایزه‌ی Netflix (۲۰۰۶ تا ۲۰۰۹) معروف شد و هنوز اول درس داده می‌شود، چون ساده‌ترین راه برای فهمیدن ایده‌هاست.

۰.۶ نقشه‌ی سه هفته: هر روش، جواب ضعف روش قبلی

1. **Baselines (هفته‌ی ۱):** ساده‌ترین جواب‌ها (میانگین، محبوبیت). شخصی‌سازی ندارند. ← Terminator را به Ana پیشنهاد می‌دهند (اشتباه).
2. **kNN-CF (هفته‌ی ۲):** «فیلم‌های شبیه به آن‌هایی که دوست داشتی». شخصی‌سازی دارد. ← با similarity درست، جواب درست می‌دهد. ولی هزینه‌اش $O(n^2)$ است و با sparsity ضعیف می‌شود.
3. **MF و LightGCN (هفته‌ی ۲):** برای هر کاربر و هر فیلم یک بردار کوتاه یاد می‌گیریم. فشرده و قوی است و با implicit کار می‌کند. ولی **فیلم جدید** را نمی‌بیند.
4. **Content-based (هفته‌ی ۳):** از محتوای خود فیلم (ژانر، متن) استفاده می‌کند. فیلم جدید را هم پیشنهاد می‌دهد. ولی کیفیت فیلم را نمی‌فهمد. ← در صنعت ترکیب (hybrid) می‌شوند.

فصل ۱ — Baselines (هفته‌ی ۱)

۱.۱ مسئله: روز اول، بدون هیچ مدلی

فردا سرویس راه می‌افتد. هنوز هیچ مدل هوشمندی نداریم، ولی سه چیز باید از روز اول کار کند:

1. **کاربر جدید** وارد می‌شود و هیچ چیز از او نمی‌دانیم. در صفحه‌ی اولش چه نشان بدهیم؟ (این مسئله‌ی **cold start** است.)
2. **صفحه‌ی هر فیلم** یک «۴٫۲ ستاره» نشان می‌دهد. کدام عدد؟ و برای فیلمی که فقط دو امتیاز دارد، منصفانه است؟
3. **ماه بعد مدل هوشمند می‌سازیم.** چطور ثابت کنیم که واقعاً بهتر از «هیچ کار هوشمندی نکردن» است؟

Baseline جواب هر سه است: ساده‌ترین روشی که بدون یادگیری، نتیجه‌ی معقول می‌دهد.

چرا baseline فقط یک تمرین کلاسی نیست:

- **معیار مقایسه است.** «MAE = ۰٫۷۵» به‌تنهایی معنی ندارد. «MAE = ۰٫۷۵ در حالی که پیش‌بینی همیشه‌ی میانگین ۰٫۸۴ می‌دهد» معنی دارد. مدل پیچیده‌ای که baseline را شکست ندهد، deploy نمی‌شود.
- **راه‌حل cold start است.** ردیف‌های «Trending» و «Most popular» در Netflix و YouTube همین baseline ها هستند.
- **fallback است.** وقتی مدل اصلی جواب ندارد (کالای جدید، کاربر جدید)، سیستم به baseline برمی‌گردد. کد استاد هم همین کار را با `.get(movie_id, avg_rating)` می‌کند.
- **feature است.** در مدل‌های رتبه‌بندی واقعی، «محبوبیت کالا» و «میانگین امتیاز کالا» به‌عنوان ورودی استفاده می‌شوند.

۱.۲ Baseline های پیش‌بینی امتیاز

۱.۲.۱ پیش‌بینی تصادفی (Random)

ایده. یک عدد تصادفی بین ۱ تا ۵ حدس بزن. این «بدترین روش معقول» است و فقط برای این است که بدانیم کف کجاست.

مثال ۱. امتیازهای واقعی $[4, 2, 5]$ و حدس‌های تصادفی $[1, 3, 5]$ است.

خطاها: $|4-1| = 3$، $|2-3| = 1$، $|5-5| = 0$. میانگین خطا (MAE) $= (3+1+0)/3 = 1.33$.

مثال ۲ (خطای مورد انتظار). اگر امتیاز واقعی ۳ باشد و حدس به‌طور یکنواخت یکی از $\{1, 2, 3, 4, 5\}$ باشد، خطاها $2, 1, 0, 1, 2$ هستند و میانگین‌شان $6/5 = 1.2$ است. اگر امتیاز واقعی ۵ باشد، خطاها $4, 3, 2, 1, 0$ و میانگین $10/5 = 2.0$ است.

پس خطای random برای امتیازهای وسط کمتر و برای امتیازهای کناری بیشتر است. روی MovieLens، استاد MAE حدود **۱٫۴۸** گرفت.

۱.۲.۲ میانگین کل (Global mean)

ایده. کاربر و فیلم را نادیده بگیر و میانگین همه‌ی امتیازها را پیش‌بینی کن:

$$\mu = \frac{1}{|R|} \sum_{(u,i) \in R} r_{ui}$$

چرا میانگین و نه عدد دیگر؟ اگر مجبور باشی برای همه یک عدد واحد پیش‌بینی کنی، میانگین عددی است که مجموع مربع خطاها را کمینه می‌کند. پس بهترین حدسِ «هیچ چیز نمی‌دانم» است.

مثال ۱ (جدول کوچک). جمع امتیازها سطر به سطر: $\text{Ana} = 5+4+1 = 10$، $\text{Ben} = 1+5+4 = 10$، $\text{Cem} = 4+5+2+3 = 14$، $\text{Dee} = 2+4+5+1 = 12$.

$$\mu = \frac{10+10+14+12}{14} = \frac{46}{14} = 3.29$$

پیش‌بینی برای (Ana → Terminator) برابر 3.29 است (و برای همه‌ی خانه‌های دیگر هم همین).

مثال ۲. امتیازهای train یک سرویس کوچک: $4, 5, 3, 4, 4, 2$. پس $\mu = 22/6 = 3.67$ و برای هر کاربر و هر فیلمی که امتیاز ندارد، 3.67 پیش‌بینی می‌شود.

کاربرد واقعی: fallback نهایی وقتی هم کاربر و هم کالا جدیدند؛ **مرکزسازی (centering)**: تقریباً هر مدلی اول μ را کم می‌کند و فقط انحراف از آن را یاد می‌گیرد؛ و «prior» در میانگین میراشده (بخش ۱.۴).

۱.۲.۳ میانگین کاربر (User mean)

ایده. بعضی آدم‌ها دست‌ودل‌بازند و به همه چیز ۴ و ۵ می‌دهند و بعضی سخت‌گیرند. میانگین خود کاربر را پیش‌بینی کن:

$$\bar{r}_u = \frac{1}{|I_u|} \sum_{i \in I_u} r_{ui}$$

که I_u مجموعه‌ی فیلم‌هایی است که کاربر u امتیاز داده.

مثال ۱ (جدول کوچک).

کاربر	امتیازها	جمع	تعداد	$\bar{r}_u$
Ana	$5, 4, 1$	10	3	3.33
Ben	$1, 5, 4$	10	3	3.33
Cem	$4, 5, 2, 3$	14	4	3.50
Dee	$2, 4, 5, 1$	12	4	3.00

پیش‌بینی برای (Ana → Terminator) برابر 3.33 است.

مثال ۲ (تفسیر). Cem با میانگین ۳٫۵ نسبت به Dee با ۳٫۰ «دست‌ودل‌بازتر» است. پس اگر Cem به فیلمی ۴ بدهد، کمتر از Dee ۴ معنی دارد: Cem ۴ فقط 0.5 بالای عادتش است، ولی Dee ۴ یک ستاره‌ی کامل بالای عادتش.

چه چیزی را می‌گیرد و چه چیزی را نه؟ «مقیاس شخصی» کاربر را می‌گیرد، ولی فیلم را کاملاً نادیده می‌گیرد: برای Ana به همه‌ی فیلم‌های ندیده همان ۳٫۳۳ را می‌دهد. پس **نمی‌تواند فیلم‌ها را برای یک کاربر رتبه‌بندی کند** و برای Top-N بی‌فایده است.

کاربرد واقعی: نرمال‌سازی کاربران. کم کردن میانگین کاربر همه را روی یک مقیاس می‌آورد؛ این دقیقاً قدم **mean-centering** در adjusted cosine (فصل ۲) است که آن‌جا حیاتی می‌شود. همچنین fallback برای کالای جدید.

۱.۲.۴ میانگین کالا (Item mean)

ایده. بعضی فیلم‌ها ذاتاً محبوب‌ترند. میانگین امتیاز خود فیلم را پیش‌بینی کن:

$$\bar{r}_i = \frac{1}{|U_i|} \sum_{u \in U_i} r_{ui}$$

مثال ۱ (جدول کوچک)، ستون به ستون.

فیلم	امتیازها	جمع	تعداد	$\bar{r}_i$
Toy Story	$5, 1, 4$	10	3	3.33
Nemo	$4, 5, 2$	11	3	3.67
Matrix	$1, 5, 4$	10	3	3.33
Terminator	$4, 2, 5$	11	3	3.67
Titanic	$3, 1$	4	2	2.00

پیش‌بینی برای (Ana → Terminator) برابر 3.67 است.

مثال ۲ (fallback، همان کاری که کد استاد می‌کند). فیلم جدیدی به نام Cars در داده‌ی train هیچ امتیازی ندارد. چون کلید Cars را پیدا نمی‌کند، `item_averages.get(Cars, avg_rating)` $\mu = 3.29$ را برمی‌گرداند.

کاربرد واقعی: عدد «۴٫۲ ★» در صفحه‌ی هر محصول (Amazon، IMDb، Google Maps) یک item mean است، معمولاً از نوع میراشده؛ و fallback برای کاربر جدید.

۱.۲.۵ Bias baseline: ترکیب هر دو

ایده. از میانگین کل شروع کن و دو اصلاح اضافه کن:

$$\hat{r}_{ui} = \mu + b_u + b_i$$

- b_u: کاربر چقدر بالاتر یا پایین‌تر از میانگین نمره می‌دهد.
- b_i: فیلم چقدر بالاتر یا پایین‌تر از میانگین نمره می‌گیرد.

نسخه‌ی ساده: $b_u = \bar{r}_u - \mu$ و $b_i = \bar{r}_i - \mu$.

چرا بهتر است؟ یک «۴» می‌تواند از «کاربر دست‌ودل‌باز + فیلم متوسط» بیاید یا از «کاربر سخت‌گیر + فیلم عالی». bias baseline این دو اثر را از هم جدا می‌کند.

مثال ۱ (Ana → Terminator).

- $b_{Ana} = 3.33 - 3.29 = +0.05$ (Ana تقریباً مثل کاربر متوسط نمره می‌دهد)
- $b_{Terminator} = 3.67 - 3.29 = +0.38$ (Terminator بالاتر از متوسط نمره گرفته)

$$\hat{r} = 3.29 + 0.05 + 0.38 = 3.71$$

مثال ۲ (Dee → Toy Story). $b_{Dee} = 3.00 - 3.29 = -0.29$ و $b_{ToyStory} = 3.33 - 3.29 = +0.05$.

$$\hat{r} = 3.29 - 0.29 + 0.05 = 3.05$$

Dee سخت‌گیر است، پس پیش‌بینی پایین‌تر از میانگین کل می‌شود.

۱.۲.۶ Damping (میرا کردن یا shrinkage)

مسئله. Titanic فقط دو امتیاز دارد. آیا واقعاً یک فیلم «۲٫۰» است، یا با دو بیننده بدشانسی آوردیم؟ با تعداد کم نباید کاملاً به میانگین اعتماد کرد.

راه‌حل. در مخرج به‌جای «تعداد»، «تعداد + λ» بگذار:

$$b_i = \frac{\sum_{u \in U_i} (r_{ui} - \mu)}{\lambda + |U_i|}$$

چرا این کار درست است؟ وقتی $|U_i|$ کوچک است، λ مخرج را خیلی بزرگ‌تر می‌کند و b_i به سمت صفر (یعنی به سمت μ) کشیده می‌شود. وقتی $|U_i|$ بزرگ است (مثلاً ۱۰۰۰)، اضافه شدن λ تقریباً هیچ اثری ندارد. **یعنی: نمونه‌ی کوچک به سمت میانگین کشیده می‌شود، نمونه‌ی بزرگ دست‌نخورده می‌ماند.**

مثال ۱ (Titanic، $\lambda = 2$). انحراف‌ها از μ: $(3 - 3.29) + (1 - 3.29) = -2.57$.

- بدون damping: $b_i = -2.57/2 = -1.29$
- با damping: $b_i = -2.57/(2+2) = -0.64$

اصلاح نصف شد، چون شواهد ضعیف است.

مثال ۲ (مقایسه‌ی کم و زیاد، $\mu = 3.5$، $\lambda = 4$).

- فیلم A: یک امتیاز ۵. انحراف 1.5. بدون damping: $b = 1.5$. با damping: $b = 1.5/(4+1) = 0.30$.
- فیلم B: ۱۰۰ امتیاز با میانگین ۴٫۵. جمع انحراف‌ها $100 \times 1.0 = 100$. بدون damping: $b = 1.0$. با damping: $b = 100/104 = 0.96$.

A که فقط یک رأی داشت از ۱٫۵ به ۰٫۳ آمد؛ B تقریباً تغییری نکرد.

کاربرد واقعی: bias baseline با damping، baseline استاندارد rating prediction است. اولین جزء راه‌حل‌های جایزه‌ی Netflix بود، در کتابخانه‌ی Surprise با اسم `BaselineOnly` آمده، و مدل‌های Matrix Factorization دقیقاً همین جمله‌ها را دارند: $\hat{r}_{ui} = \mu + b_u + b_i + \mathbf{p}_u^\top \mathbf{q}_i$. روی MovieLens، bias baseline بهترین baseline است (MAE حدود ۰٫۶۸ در برابر ۰٫۸۴ برای میانگین کل).

۱.۲.۷ جمع‌بندی: baseline ها درباره‌ی سوال ما چه می‌گویند؟

روش	پیش‌بینی Ana → Terminator	پیشنهاد بدهیم؟
میانگین کل	3.29	شاید
میانگین کاربر	3.33	شاید
میانگین فیلم	3.67	بله
Bias baseline	3.71	بله

هر چهار روش می‌گویند Ana احتمالاً Terminator را دوست دارد، و هر چهار اشتباه می‌کنند. هیچ‌کدام نگاه نمی‌کنند Ana **کدام** فیلم‌ها را دوست داشته، پس نمی‌توانند بفهمند او فیلم اکشن دوست ندارد. این دقیقاً ضعفی است که فصل ۲ برطرف می‌کند.

۱.۳ اندازه‌گیری: MAE و RMSE

موقعیت. هفته‌ی بعد سه امتیاز جدید می‌رسد. این‌ها **test set** هستند: امتیازهایی که مدل موقع ساخته شدن ندیده است.

$$\text{MAE} = \frac{1}{|T|}\sum_{(u,i)\in T}|r_{ui}-\hat{r}_{ui}| \qquad \text{RMSE} = \sqrt{\frac{1}{|T|}\sum_{(u,i)\in T}(r_{ui}-\hat{r}_{ui})^2}$$

چرا دو معیار؟ در RMSE خطا به توان ۲ می‌رسد، پس **خطاهای بزرگ خیلی بیشتر جریمه می‌شوند**.

- MAE وقتی مناسب است که هر ستاره خطا هزینه‌ی برابر دارد؛ توضیحش هم راحت است («به‌طور متوسط ۱٫۲ ستاره اشتباه می‌کنیم»).
- RMSE وقتی مناسب است که اشتباه بزرگ بدتر است (پیشنهاد چیزی که کاربر از آن متنفر است بدتر از دو بار کمی اشتباه کردن است).
- **همیشه** $\text{RMSE} \geq \text{MAE}$. فقط وقتی برابرند که همه‌ی خطاها اندازه‌ی یکسان داشته باشند.

مثال ۱ (baseline میانگین کاربر روی test). امتیازهای واقعی جدید: (Ana → Terminator) برابر 1، (Ben → Nemo) برابر 2 و (Dee → Toy Story) برابر 3.

مورد	واقعی	پیش‌بینی	خطای مطلق	مربع خطا
Ana, Terminator	1	3.33	2.33	5.44
Ben, Nemo	2	3.33	1.33	1.78
Dee, Toy Story	3	3.00	0.00	0.00

$\text{MAE} = (2.33+1.33+0)/3 = 1.22$ و $\text{RMSE} = \sqrt{(5.44+1.78+0)/3} = \sqrt{2.41} = 1.55$.

خطای بزرگ (۲٫۳۳) در RMSE سهم ۷۵٪ دارد $(5.44/7.22)$ ولی در MAE فقط ۶۴٪ $(2.33/3.67)$.

مثال ۲. امتیازهای واقعی $[4, 2, 5, 3]$ و پیش‌بینی ثابت 3.5.

خطاها: $0.5,\ 1.5,\ 1.5,\ 0.5 \leftarrow \text{MAE} = 4/4 = 1.0$.

مربع‌ها: $0.25,\ 2.25,\ 2.25,\ 0.25 \leftarrow$ میانگین $= 1.25 \leftarrow \text{RMSE} = \sqrt{1.25} = 1.118$.

مثال ۳. خطاها $[0, 1, 3]$: $\text{MAE} = 4/3 = 1.33$ و $\text{RMSE} = \sqrt{(0+1+9)/3} = \sqrt{3.33} = 1.83$.

Train/Test و Data leakage

قانون: همه‌ی میانگین‌ها و همه‌ی آماره‌ها فقط از داده‌ی **train** حساب می‌شوند.

چرا؟ اگر امتیازهای test در محاسبه‌ی μ یا $\bar{r}_u$ وارد شوند، مدل بخشی از جواب را «از قبل می‌داند» و خطای اندازه‌گیری‌شده بهتر از واقعیت به نظر می‌رسد. اسمش **data leakage** است و رایج‌ترین دلیل این است که مدلی در آزمایش عالی است و در محیط واقعی شکست می‌خورد.

مثال ۱. در جدول کوچک، میانگین train $= 46/14 = 3.29$ است. اگر اشتباهاً سه امتیاز (۳، ۲، ۱) test هم وارد شوند: $\mu = (46+6)/17 = 3.06$. پیش‌بینی برای test به جواب‌ها نزدیک‌تر شده، فقط چون جواب‌ها را دیده‌ایم.

مثال ۲ (کد استاد در Top-N). استاد آیتم مثبت را از کل `ratings` انتخاب می‌کند ولی popularity را از `X_train` (۹۰٪ داده) حساب می‌کند. پس همان امتیازی که داریم تست می‌کنیم، معمولاً در شمارش popularity هم آمده است. راه درست: آیتم test را از train حذف کن (leave-one-out).

۱.۴ Baseline های Top-N: صفحه‌ی اول را چه بگذاریم؟

حالا وظیفه‌ی دوم. نمره پیش‌بینی نمی‌کنیم؛ باید **انتخاب کنیم چه نشان بدهیم**. به هر کالا یک **score** می‌دهیم و کالاهای با بیشترین score را نشان می‌دهیم.

روش	score	ضعف
Random	عدد تصادفی	فقط کف مقایسه
Popularity	تعداد امتیازها	تعامل را می‌شمارد نه رضایت را؛ به همه یک لیست می‌دهد
Highest average	میانگین امتیاز	فیلم با یک امتیاز ۵ اول می‌شود
Damped average	$\frac{\sum r + \lambda\mu}{n + \lambda}$	راه‌حل مشکل بالا (همین IMDb Top 250 است)

چرا popularity کار می‌کند؟ بیشتر آدم‌ها چیزی را دوست دارند که بیشتر آدم‌ها دوست دارند؛ برای کسی که هیچ چیز از او نمی‌دانیم، کالای محبوب حدس امنی است.

چرا highest average ضعیف است؟ میانگینِ یک یا دو امتیاز قابل اعتماد نیست. روی داده‌ی واقعی، بالای این لیست پر از فیلم‌های ناشناخته با یکی دو امتیاز کامل است.

damped average چه می‌کند؟ فرض می‌کند هر فیلم λ امتیاز خیالی برابر μ دارد. همان ایده‌ی shrinkage بخش ۱.۲.۶.

مثال ۱. چهار فیلم از یک کاتالوگ بزرگ ($\mu = 3.5$، $\lambda = 10$):

فیلم	تعداد	میانگین	جمع امتیازها	محاسبه‌ی damped	score
Blockbuster	200	4.2	840	$(840+35)/210$	4.17
Classic	150	4.5	675	$(675+35)/160$	4.44
Niche gem	1	5.0	5	$(5+35)/11$	3.64
Flop	80	2.1	168	$(168+35)/90$	2.26

- Popularity: Blockbuster، Classic، **Flop**، Niche gem — Flop سوم است با این‌که مردم از آن بدشان می‌آید.
- Highest average: **Niche gem**، Classic، Blockbuster، Flop — یک رأی، اولش کرده.
- Damped: Classic، Blockbuster، Niche gem، Flop — منطقی.

مثال ۲ (جدول کوچک، $\mu = 3.29$، $\lambda = 2$).

فیلم	تعداد	جمع	damped $= \frac{\text{sum}+2\times3.29}{n+2}$
Toy Story	3	10	$16.57/5 = 3.31$
Nemo	3	11	$17.57/5 = 3.51$
Matrix	3	10	3.31
Terminator	3	11	3.51
Titanic	2	4	$10.57/4 = 2.64$

میانگین ساده‌ی Titanic ۲٫۰ بود؛ چون فقط دو رأی دارد، به ۲٫۶۴ (نزدیک μ) کشیده شد.

۱.۵ اندازه‌گیری Top-N: Hit Ratio

برای Top-N نمی‌توان MAE گرفت، چون نمره‌ای پیش‌بینی نمی‌شود. سوال این است: **آیا فیلمی که کاربر واقعاً دوست داشت در لیست ما هست؟**

پروتکل استاد: «۱ مثبت + ۹۹ منفی». برای هر کاربر:

1. یک فیلم که کاربر به آن ۵ داده انتخاب کن: **مثبت (positive)**.
2. ۹۹ فیلم تصادفی که کاربر هرگز امتیاز نداده انتخاب کن: **منفی (negative)**.
3. هر ۱۰۰ فیلم را با روش توصیه score بده و مرتب کن.
4. اگر مثبت جزو K تای اول بود، یک **hit** حساب کن.

$$\text{HR@}K = \frac{\text{number of hits}}{\text{number of users tested}}$$

HR یک روش تصادفی چقدر است؟ مثبت با احتمال برابر در هر کدام از ۱۰۰ جایگاه قرار می‌گیرد، پس احتمال این‌که در K جایگاه اول باشد $K/100$ است:

$$\text{HR@}K_{\text{random}} = \frac{K}{100} \;\Rightarrow\; \text{HR@5} = 0.05,\ \text{HR@10} = 0.10,\ \text{HR@20} = 0.20$$

این را حفظ کن؛ «سطح صفر» این پروتکل است و سوال امتحانی محتملی است.

چرا ۹۹ منفی و نه کل کاتالوگ؟ امتیاز دادن و مرتب‌کردن همه‌ی کالاها برای همه‌ی کاربران گران است؛ sampling ارزان‌ترش می‌کند (این پروتکل با مقاله‌ی NCF رایج شد).

مثال ۱ (۱ مثبت + ۴ منفی، popularity، $K = 2$). اعداد تعداد امتیازها هستند.

کاربر	مثبت (تعداد)	منفی‌ها	مرتب‌شده	رتبه‌ی مثبت	hit@2؟
U1	Blockbuster (200)	$3, 150, 1, 12$	$\mathbf{200}, 150, 12, 3, 1$	1	بله
U2	Niche gem (1)	$80, 2, 0, 5$	$80, 5, 2, \mathbf{1}, 0$	4	خیر
U3	Classic (150)	$200, 7, 1, 80$	$200, \mathbf{150}, 80, 7, 1$	2	بله

$\mathrm{HR@2} = 2/3 = 0.67$. روش تصادفی با ۵ کاندید: $2/5 = 0.40$.

به U2 دقت کن: popularity برای کسی که سلیقه‌ی خاص دارد شکست می‌خورد، و این در میانگین HR دیده نمی‌شود.

مثال ۲ (سوال Q5 استاد). Top-3 برای ۵ کاربر:

کاربر	فیلم هدف	Top-3	نتیجه
User 1	Movie 10	[22 ,10 ,5]	hit
User 2	Movie 45	[3 ,89 ,12]	miss
User 3	Movie 101	[74 ,2 ,101]	hit
User 4	Movie 12	[90 ,12 ,14]	hit
User 5	Movie 8	[150 ,88 ,9]	miss

$\mathrm{HR} = 3/5 = 0.60$.

چرا popularity در کد استاد HR@20 بالای ۰٫۹۱ گرفت؟ بخش بزرگی از این عدد از **پروتکل** می‌آید، نه از قدرت popularity: مثبت فیلمی است که کاربر واقعاً دیده (پس معمولاً محبوب است)، ولی ۹۹ منفی به‌طور یکنواخت از کل کاتالوگ می‌آیند که نیمی از آن فقط ۱ یا ۲ امتیاز دارد. وقتی منفی‌ها را **متناسب با محبوبیت** انتخاب کنیم (رقبای واقعی‌تر)، HR@20 popularity از حدود ۰٫۸۸ به ۰٫۴۷ می‌رسد.

درس: عدد offline فقط همراه با پروتکلش معنی دارد. HR های اندازه‌گیری‌شده با روش‌های متفاوت را هیچ‌وقت مقایسه نکن.

۱.۶ الگوریتم و پیچیدگی

```
FIT_BASELINES(train): # one pass over the data: O(|R|) mu <- mean of all ratings user_avg[u] <- mean of the ratings of user u item_avg[i] <- mean of the ratings of item i pop[i] <- number of ratings of item i PREDICT_ITEM_AVG(u, i): # O(1) if i in item_avg: return item_avg[i] else: return mu # fallback for an unseen item EVALUATE_HIT_RATIO(score, users, K): hits <- 0 for each user u: pos <- one held-out item u liked # must NOT be in the training data negs <- 99 random items u has NOT interacted with rank the 100 candidates by score, highest first if pos is within the first K: hits <- hits + 1 return hits / number of users
```

ساختن جدول‌ها یک بار گذشتن از داده است: $O(|R|)$. هر پیش‌بینی یک lookup است: $O(1)$. برای همین baseline ها عملاً رایگان‌اند و همیشه به‌عنوان fallback در دسترس‌اند.

۱.۷ کد استاد و ایرادهایش

کد استاد سه بخش دارد: (۱) پیش‌بینی تصادفی روی همه‌ی امتیازها؛ (۲) `train_test_split(test_size=0.1)`، ساختن جدول میانگین‌ها با `groupby('movieId')['rating'].mean()` و پیش‌بینی با `.get(movie_id, avg_rating)`؛ (۳) حلقه‌ی Hit Ratio، سه بار برای سه score.

ایرادها (منبع سوال‌های «این کد چه مشکلی دارد؟»):

1. حلقه‌ی Python با `.iloc` / `.iterrows()` روی ردیف‌ها خیلی کند است؛ در عمل با یک خط vectorized نوشته می‌شود (`fillna` + `map`).
2. حلقه‌ی ۴۰ خطی evaluation سه بار کپی شده؛ باید یک تابع باشد که روش امتیازدهی را به‌عنوان ورودی بگیرد.
3. random روی کل داده ارزیابی شده ولی بقیه روی test؛ مقایسه روی داده‌ی یکسان نیست.
4. random فقط اعداد صحیح ۱ تا ۵ می‌سازد، ولی امتیازهای واقعی نیم‌ستاره هم دارند (۰٫۵ تا ۵).
5. **leakage** در Top-N (مثال ۲ بخش ۱.۳).
6. میانگین ساده برای «highest average» به جای میانگین میراشده.

۱.۸ سوال‌های امتحان — فصل ۱

سوال (از نتیجه‌گیری نوت‌بوک استاد). چرا popularity baseline قوی است و highest average ضعیف؟

popularity: داده‌ی تعامل توزیع long-tail دارد؛ تعداد کمی کالا بیشتر تعامل‌ها را دارند، پس آیتم مثبتِ نگه‌داشته‌شده به احتمال زیاد محبوب است. در پروتکل ۱+۹۹، منفی‌ها بیشتر کالاهای کم‌طرفدارند، پس رتبه‌بندی بر اساس تعداد خیلی موفق است. **highest average:** میانگینِ تعداد خیلی کم امتیاز قابل اعتماد نیست؛ فیلمی با یک امتیاز ۵ بیشترین score را می‌گیرد. راه‌حل: حداقل تعداد امتیاز، یا میانگین میراشده.

English: Interaction data is long-tailed, so a held-out positive is very likely a popular item, while uniformly sampled negatives are mostly long-tail items; ranking by count is therefore very effective. Highest average is unreliable because means of very few ratings are noisy: an item with one 5-star rating ranks first. Fix: minimum support or a damped average $(\sum r + \lambda\mu)/(n+\lambda)$.

سوال. HR@10 یک recommender تصادفی در پروتکل ۱+۹۹ چقدر است؟ $10/100 = 0.10$، چون مثبت با احتمال برابر در هر کدام از ۱۰۰ جایگاه قرار می‌گیرد.

سوال. MAE و RMSE؛ کدام بزرگ‌تر است و چه فرقی دارند؟ همیشه RMSE ≥ MAE. RMSE به خاطر توان ۲ خطاهای بزرگ را بیشتر جریمه می‌کند؛ MAE همه‌ی خطاها را خطی وزن می‌دهد و به outlier ها مقاوم‌تر است.

سوال. چرا میانگین‌ها فقط از train حساب می‌شوند؟ چون وارد شدن امتیازهای test، جواب را به مدل نشان می‌دهد (data leakage) و خطا را خوش‌بینانه می‌کند.

سوال. برای کاربر یا کالایی که در train نیست چه می‌شود؟ میانگینی برایش وجود ندارد (cold start)؛ کد به میانگین کل برمی‌گردد (`.get(id, avg_rating)`).

سوال. ضعف اصلی popularity با وجود HR بالا چیست؟ شخصی‌سازی ندارد: همه یک لیست می‌گیرند؛ coverage و novelty نزدیک صفر است و popularity bias را تقویت می‌کند (پولدار، پولدارتر می‌شود).

سوال (الگوریتمی). شبه‌کد ارزیابی با HR@K را بنویس. شبه‌کد بخش ۱.۶. سه نکته نباید جا بیفتد: منفی‌ها از کالاهای **امتیازنداده** انتخاب می‌شوند؛ مثبت باید از train **حذف** شده باشد؛ مخرج **تعداد test case ها** است.

فصل ۲ — Collaborative Filtering مبتنی بر همسایگی (هفته‌ی ۲، بخش اول)

۲.۱ مسئله: «چون X را دیدی...»

حالا کاربر و امتیاز داریم. مدیر محصول دو چیز می‌خواهد:

1. یک ردیف در صفحه‌ی اول: **«چون Toy Story را دیدی»**.
2. پیشنهادهای بد کمتر: *«Ana به Matrix یک ستاره داد و ما باز Terminator را به او نشان دادیم.»*

baseline های فصل ۱ هیچ‌کدام را نمی‌توانند، چون هیچ‌وقت نگاه نمی‌کنند کاربر **کدام** کالاها را دوست داشته.

همین مسئله در محصولات واقعی: «Customers who bought this item also bought» در Amazon (اولین کاربرد؛ آمازون روشش را در ۲۰۰۳ منتشر کرد و نوت‌بوک استاد همان مقاله را ارجاع می‌دهد)؛ ردیف «Because you watched» در Netflix؛ «Fans also like» در Spotify؛ «Similar products» در هر فروشگاه اینترنتی.

ایده: Collaborative Filtering (CF)

فقط از خود جدول امتیازها استفاده کن؛ هیچ ژانر، توضیح یا بازیگری لازم نیست. فرض اصلی:

> کسانی که در گذشته هم‌سلیقه بوده‌اند، در آینده هم هم‌سلیقه خواهند بود.

ما هیچ‌جا به سیستم نگفتیم Matrix و Terminator هر دو اکشن‌اند. سیستم این را فقط از این‌جا می‌فهمد که **همان آدم‌ها هر دو را دوست دارند**. معنی «collaborative» همین است، و برای همین CF برای هر نوع کالایی (فیلم، آهنگ، کفش) کار می‌کند بدون این‌که کالا را «بفهمد».

	User-based CF	Item-based CF (روش استاد)
سوال	کدام کاربرها شبیه Ana هستند و آن‌ها چه دوست داشتند؟	کدام فیلم‌ها شبیه فیلم‌هایی‌اند که Ana دوست داشت؟
شباهت بین	دو سطر جدول	دو ستون جدول

> **چرا صنعت item-based را انتخاب کرد؟**
>
> 1. **پایداری:** سطر یک کاربر با هر کلیک عوض می‌شود، ولی رابطه‌ی «Matrix شبیه Terminator است» آهسته تغییر می‌کند. پس شباهت کالاها را می‌شود یک بار (مثلاً شب‌ها) حساب کرد و تمام روز استفاده کرد.
> 2. **توضیح‌پذیری:** «چون Toy Story را دیدی» دلیلی است که کاربر می‌فهمد و به آن اعتماد می‌کند.
> 3. **اندازه:** بیشتر سرویس‌ها کاربر خیلی بیشتری از کالا دارند، پس جدول کالا–کالا کوچک‌تر است.

کِی kNN-CF ابزار درست است: داده‌ی تعامل کافی داری، یک روش قوی و سریع بدون training می‌خواهی، و توضیح‌پذیری مهم است. **کِی شکست می‌خورد:** کالا یا کاربر جدید بدون تعامل (cold start)، داده‌ی خیلی sparse (هم‌پوشانی کم ← شباهت غیرقابل اعتماد)، و scale خیلی بزرگ ($O(n^2)$).

۲.۲ برنامه‌ی کار

برای پیش‌بینی (Ana → Terminator):

1. اندازه بگیر Terminator چقدر شبیه هر فیلمی است که Ana قبلاً امتیاز داده (Toy Story، Nemo، Matrix).
2. امتیازهای Ana به آن فیلم‌ها را میانگین بگیر، طوری که فیلم‌های شبیه‌تر به Terminator وزن بیشتری داشته باشند.

منطق: *اگر Terminator خیلی شبیه Matrix باشد، نظر Ana درباره‌ی Matrix، نظرش درباره‌ی Terminator را نشان می‌دهد.*

همه چیز به قدم ۱، یعنی **similarity**، بستگی دارد. سه روش را می‌بینیم و خواهیم دید سه جواب متفاوت می‌دهند.

۲.۳ Similarity اول: Jaccard

ایده. مقدار امتیاز را نادیده بگیر. فقط بپرس: چه *کسانی این فیلم را امتیاز داده‌اند؟* دو فیلم شبیه‌اند اگر همان آدم‌ها امتیازشان داده باشند.

$$J(i,j) = \frac{|U_i \cap U_j|}{|U_i \cup U_j|} = \frac{\text{users who rated both}}{\text{users who rated at least one}}$$

یک میان‌بر مفید: $|U_i \cup U_j| = |U_i| + |U_j| - |U_i \cap U_j|$. مقدار Jaccard همیشه بین ۰ و ۱ است.

مجموعه‌ی کاربران هر فیلم (ستون‌های جدول را بخوان):

فیلم	کاربرانی که امتیاز داده‌اند
Toy Story	{Ana, Ben, Cem}
Nemo	{Ana, Cem, Dee}
Matrix	{Ana, Ben, Dee}
Terminator	{Ben, Cem, Dee}
Titanic	{Cem, Dee}

مثال ۱ (Terminator با هر فیلم Ana).

- Terminator و Toy Story: هر دو $\{Ben, Cem\}$ ← ۲ نفر؛ حداقل یکی $\{Ana, Ben, Cem, Dee\}$ ← ۴ نفر؛ $J = 2/4 = 0.50$
- Terminator و Nemo: هر دو $\{Cem, Dee\}$ ← ۲؛ حداقل یکی ۴ ← $J = 0.50$
- Terminator و Matrix: هر دو $\{Ben, Dee\}$ ← ۲؛ حداقل یکی ۴ ← $J = 0.50$

نتیجه: از نظر Jaccard، Terminator به هر سه فیلم **به یک اندازه** شبیه است. نمی‌تواند آن‌ها را از هم جدا کند، چون نگاه نمی‌کند مردم فیلم را دوست داشتند یا نه.

مثال ۲ (Q4) استاد، Task 1: Jaccard بین A و D).

	Item A	Item B	Item C	Item D
User 1	5	3	?	1
User 2	4	?	4	2
User 3	1	1	?	5
User 4	4	2	3	?

$U_A = \{1,2,3,4\}$ و $U_D = \{1,2,3\}$. اشتراک $\{1,2,3\}$ ← ۳؛ اجتماع $\{1,2,3,4\}$ ← ۴. پس $J(A,D) = 3/4 = 0.75$.

مثال ۳ (مثال خود استاد). $A = \{a,b,c,d\}$ و $B = \{b,d,e,f,g\}$. اشتراک $\{b,d\}$ ← ۲؛ اجتماع $4 + 5 - 2 = 7$. پس $J = 2/7 = 0.29$.

Jaccard کِی انتخاب درستی است؟ برای **داده‌ی implicit** که اصلاً مقدار امتیاز ندارد: خرید، کلیک، پخش. «کسانی که X را خریدند Y را هم خریدند» یک شمارش co-occurrence است و Jaccard دقیقاً یک co-occurrence نرمال‌شده است.

۲.۴ Similarity دوم: Cosine

ایده. از مقدار امتیازها استفاده کن. هر فیلم را یک بردار امتیاز در نظر بگیر و زاویه‌ی بین دو بردار را اندازه بگیر: هم‌جهت ← ۱، بی‌ربط ← ۰، مخالف ← ۱-.

$$\cos(i,j) = \frac{\sum_u r_{ui}\, r_{uj}}{\sqrt{\sum_u r_{ui}^2}\,\sqrt{\sum_u r_{uj}^2}}$$

هر جزء فرمول چه می‌کند؟

- **صورت (ضرب داخلی):** کاربر به کاربر، «امتیاز i × امتیاز j» را جمع می‌زند. وقتی همان کاربران به هر دو امتیاز بالا بدهند، بزرگ می‌شود. اگر یک کاربر به یکی امتیاز مثبت و به دیگری منفی بدهد، حاصل‌ضرب منفی است و از شباهت کم می‌کند.
- **مخرج (طول دو بردار):** نرمال‌سازی است؛ نمی‌گذارد فیلمی فقط به خاطر اعداد بزرگ‌تر یا تعداد امتیاز بیشتر، به همه شبیه به نظر برسد. نتیجه بین ۱- و ۱ می‌ماند.

جمع روی کدام کاربران؟ استاد فقط کاربرانی را می‌گیرد که **هر دو** فیلم را امتیاز داده‌اند (co-rated) — با `np.intersect1d`. این جزئیات، کلید جواب سوال «Cosine در برابر Jaccard» استاد است (بخش ۲.۱۱).

مثال ۱ (Terminator و Toy Story). کاربران مشترک: Ben و Cem. Terminator $= [4,2]$ و Toy Story $= [1,4]$.

- ضرب داخلی: $4 \times 1 + 2 \times 4 = 4 + 8 = 12$
- طول Terminator: $\sqrt{16+4} = \sqrt{20} = 4.47$؛ طول Toy Story: $\sqrt{1+16} = \sqrt{17} = 4.12$
- $\cos = 12/(4.47 \times 4.12) = 12/18.44 = 0.65$

به همین ترتیب: Terminator و Nemo (کاربران Cem، Dee: $[2,5]$ و $[5,2]$): $\cos = 20/29 = 0.69$. Terminator و Matrix (Ben، Dee: $[4,5]$ و $[5,4]$): $\cos = 40/41 = 0.98$.

مثال ۲ (Q4 استاد، Task 2: Cosine بین A و D). کاربران مشترک ۱، ۲، ۳: $\vec{A} = [5, 4, 1]$ و $\vec{D} = [1, 2, 5]$.

- ضرب داخلی: $5 \times 1 + 4 \times 2 + 1 \times 5 = 18$
- طول‌ها: $\sqrt{25 + 16 + 1} = \sqrt{42} = 6.48$ و $\sqrt{1 + 4 + 25} = \sqrt{30} = 5.48$
- $\cos = 18 / (6.48 \times 5.48) = 18 / 35.50 = 0.507$

مثال ۳ (مثال کد استاد با `intersect1d`). کاربران i: $[1, 3, 6, 9, 12, 15]$ با امتیازهای $[4, 3, 2, 4, 2, 5]$؛ کاربران j: $[1, 6, 8, 12]$ با امتیازهای $[2, 4, 2, 5]$.

- کاربران مشترک: $[1, 6, 12]$؛ اندیس‌ها در i: $[0, 2, 4]$ و در j: $[0, 1, 3]$
- بردارهای هم‌تراز: $[4, 2, 2]$ و $[2, 4, 5]$
- ضرب داخلی: $8 + 8 + 10 = 26$؛ طول‌ها $\sqrt{24} = 4.90$ و $\sqrt{45} = 6.71$
- $\cos = 26 / 32.86 = 0.791$

مثال ۴ (مثال‌های شهودی استاد).

- هم‌جهت: $[1, 2, 3]$ و $[2, 4, 6]$: ضرب داخلی 28، طول‌ها $\sqrt{14}$ و $2\sqrt{14}$ ← $\cos = 28 / 28 = 1.0$ (بزرگی فرق دارد، جهت یکی است).
- عمود: $[1, 0]$ و $[0, 1]$ ← $0 / 1 = 0$.
- مخالف: $[1, 2]$ و $[-1, -2]$ ← $-5 / 5 = -1$.
- علامت‌ها: $[3, -2, 4]$ و $[2, 3, 5]$: ضرب‌ها $+6, \ -6, \ +20$ ← جمع 20؛ طول‌ها $\sqrt{29} = 5.39$ و $\sqrt{38} = 6.16$ ← $\cos = 20 / 33.20 = 0.602$. ژانر ۲ مخالف بود و اثر ژانر ۱ را خنثی کرد.

مشکل cosine روی امتیازهای ستاره‌ای

به مثال ۱ برگرد: **Terminator و Nemo.** Cem به Terminator ۲ و به Nemo ۵ داد. Dee به Terminator ۵ و به Nemo ۲ داد. **دقیقاً برعکس هم‌اند**: هر کس یکی را دوست دارد از دیگری بدش می‌آید. و cosine باز می‌گوید ۰٫۶۹، «نسبتاً شبیه».

چرا؟ همه‌ی امتیازها مثبت‌اند (۱ تا ۵)، پس هر حاصل‌ضرب $r_{ui} \times r_{uj}$ مثبت است و **هر کاربر به شباهت اضافه می‌کند**، حتی کاربری که مخالف است. حتی بدتر: مخالفت (۵ و ۱ ← حاصل‌ضرب ۵) بیشتر از توافقِ «هر دو بدند» (۱ و ۱ ← حاصل‌ضرب ۱) اضافه می‌کند. cosine خام روی ستاره‌ها راهی برای گفتن «این دو ضد هم‌اند» ندارد.

کجا cosine خام کاملاً درست است؟ روی داده‌ی implicit صفر و یک: $1 \times 1 = 1$ یعنی هر دو تعامل داشتند؛ بقیه صفر است؛ امتیاز منفی معنی ندارد.

۲.۵ Similarity سوم: Adjusted cosine (mean-centered)

ایده. قبل از مقایسه، هر امتیاز را به «چقدر بالاتر یا پایین‌تر از میانگین خود این کاربر» تبدیل کن:

$$\tilde{r}_{ui} = r_{ui} - \bar{r}_u$$

- عدد **مثبت** یعنی «بیشتر از معمولش دوست داشت».

- عدد **منفی** یعنی «کمتر از معمولش دوست داشت».

حالا وقتی دو کاربر مخالف‌اند، یکی مثبت و یکی منفی است، حاصل‌ضرب **منفی** می‌شود و شباهت را **کم** می‌کند. توافق و مخالفت بالاخره اثر مخالف دارند. (همین ایده با میانگین کالا به جای کاربر، **Pearson correlation** است.)

$$\text{sim}(i,j) = \frac{\sum_u (r_{ui} - \bar{r}_u)(r_{uj} - \bar{r}_u)}{\sqrt{\sum_u (r_{ui} - \bar{r}_u)^2}\,\sqrt{\sum_u (r_{uj} - \bar{r}_u)^2}} \in [-1, 1]$$

جدول centered (میانگین هر کاربر از فصل ۱ کم شده: Ana ۳٫۳۳، Ben ۳٫۳۳، Cem ۳٫۵۰، Dee ۳٫۰۰):

Titanic	Terminator	Matrix	Nemo	Toy Story	
?	?	-2.33	$+0.67$	$+1.67$	**Ana**
?	$+0.67$	$+1.67$	?	-2.33	**Ben**
-0.50	-1.50	?	$+1.50$	$+0.50$	**Cem**
-2.00	$+2.00$	$+1.00$	-1.00	?	**Dee**

از علامت‌ها می‌شود سلیقه را خواند: Ana روی انیمیشن‌ها مثبت و روی Matrix منفی است؛ Ben برعکس.

مثال ۱ (Terminator با سه فیلم Ana).

با **Toy Story** (Ben، Cem): Terminator $= [+0.67, -1.50]$ و Toy Story $= [-2.33, +0.50]$

- ضرب داخلی: $(0.67)(-2.33) + (-1.50)(0.50) = -1.56 - 0.75 = -2.31$
- طول‌ها: $\sqrt{0.67^2 + 1.50^2} = 1.64$ و $\sqrt{2.33^2 + 0.50^2} = 2.39$
- شباهت $= -2.31/(1.64 \times 2.39) = -0.59$

با **Nemo** (Cem، Dee): $[-1.50, +2.00]$ و $[+1.50, -1.00]$ ← ضرب داخلی $-4.25 = -2.25 - 2.00$؛ طول‌ها 2.50 و 1.80 ← -0.94

با **Matrix** (Ben، Dee): $[+0.67, +2.00]$ و $[+1.67, +1.00]$ ← ضرب داخلی $3.11 = 2.00 + 1.11$؛ طول‌ها 2.11 و 1.94 ← $+0.76$

مثال ۲ (Q4 استاد: adjusted cosine بین A و D).

1. میانگین کاربران: User 1: $(5+3+1)/3 = 3.00$؛ User 2: $(4+4+2)/3 = 3.33$؛ User 3: $(1+1+5)/3 = 2.33$.
2. A مرکزی: $[5-3,\ 4-3.33,\ 1-2.33] = [2.00,\ 0.67,\ -1.33]$. D مرکزی: $= [1-3,\ 2-3.33,\ 5-2.33]$ $[-2.00,\ -1.33,\ 2.67]$.
3. ضرب داخلی: $-4.00 - 0.89 - 3.56 = -8.44$. طول‌ها: 2.49 و 3.59.
4. شباهت $= -8.44/(2.49 \times 3.59) = -0.943$.

A و D ضد هم‌اند (هر که A را دوست دارد از D بدش می‌آید)، ولی Jaccard می‌گفت ۰٫۷۵ و cosine خام ۰٫۵۱.

مقایسه‌ی سه similarity

Terminator در مقایسه با	Jaccard	Cosine خام	Adjusted cosine	قضاوت انسانی
Toy Story	0.50	0.65	-0.59	متفاوت
Nemo	0.50	0.69	-0.94	ضد هم
Matrix	0.50	0.98	$+0.76$	شبیه

فقط adjusted cosine با عقل سلیم جور است. Jaccard به همه یک عدد می‌دهد؛ cosine خام به همه عدد مثبت نسبتاً بالا؛ adjusted cosine هر دو علامت را دارد و فیلم‌ها را در طرف درست می‌گذارد.

۲.۶ از similarity به پیش‌بینی

فرمول (میانگین وزن‌دار):

$$\hat{r}_{ui} = \frac{\sum_{j \in N_k(i;u)} \text{sim}(i,j) \cdot r_{uj}}{\sum_{j \in N_k(i;u)} |\text{sim}(i,j)|}$$

$N_k(i;u)$ یعنی k فیلمی از بین فیلم‌هایی که u امتیاز داده، که بیشترین شباهت را به i دارند.

چرا بر جمع شباهت‌ها تقسیم می‌کنیم؟ تا نتیجه یک میانگین واقعی باشد و روی مقیاس ۱ تا ۵ بماند. اگر همه‌ی شباهت‌ها برابر باشند، فرمول همان میانگین ساده‌ی امتیازهای کاربر را می‌دهد. **دقت کن: مخرج جمع شباهت‌هاست، نه تعداد همسایه‌ها.**

مثال ۱ (Q4 استاد، Task 3). امتیاز User 2 به Item B با $k = 2$ و Jaccard؛ داده‌شده: $J(B,A) = 0.75$، $J(B,D) = 0.5$؛ User 2 به A نمره‌ی ۴ و به D نمره‌ی ۲ داده.

- صورت: $0.75 \times 4 + 0.5 \times 2 = 3.0 + 1.0 = 4.0$
- مخرج: $0.75 + 0.5 = 1.25$
- $\hat{r} = 4.0 / 1.25 = 3.2$

مثال ۲ (Terminator ← Ana با Jaccard و cosine خام). امتیازهای Ana: Toy Story ۵، Nemo ۴، Matrix ۱.

- **Jaccard** (همه‌ی وزن‌ها ۰٫۵۰): $\hat{r} = \frac{0.5 \times 5 + 0.5 \times 4 + 0.5 \times 1}{1.5} = \frac{5.0}{1.5} = 3.33$ ← دقیقاً میانگین Ana؛ با وزن‌های برابر، روش هیچ چیزی به baseline اضافه نمی‌کند.
- **Cosine خام:** $\hat{r} = \frac{0.65 \times 5 + 0.69 \times 4 + 0.98 \times 1}{0.65 + 0.69 + 0.98} = \frac{3.25 + 2.76 + 0.98}{2.32} = \frac{6.99}{2.32} = 3.02$ ← کمی پایین‌تر، ولی هنوز «احتمالاً خوشش می‌آید».

مثال ۳ (Terminator ← Ana با adjusted cosine). وقتی شباهت روی امتیازهای centered حساب شده، پیش‌بینی هم از امتیازهای centered ساخته می‌شود: از میانگین Ana شروع کن و میانگین وزن‌دار انحراف‌هایش را اضافه کن.

$$\hat{r}_{ui} = \bar{r}_u + \frac{\sum_j \text{sim}(i,j)\,(r_{uj} - \bar{r}_u)}{\sum_j |\text{sim}(i,j)|}$$

فیلم j	شباهت با Terminator	امتیاز centered Ana	حاصل‌ضرب	معنی
Toy Story	-0.59	$+1.67$	-0.98	فیلمی را دوست داشت که **ضد** Terminator است ← پایین
Nemo	-0.94	$+0.67$	-0.63	همین ← پایین
Matrix	$+0.76$	-2.33	-1.77	از فیلمی **شبیه** Terminator بدش آمد ← پایین

جمع حاصل‌ضرب‌ها $= -3.38$؛ جمع قدرمطلق شباهت‌ها $= 2.29 = 0.59 + 0.94 + 0.76$.

$$\hat{r} = 3.33 + \frac{-3.38}{2.29} = 3.33 - 1.48 = 1.86$$

سه شاهد همه یک جهت را نشان می‌دهند و روش آن‌ها را ترکیب کرد. **این اولین روشی است که سوال ما را درست جواب می‌دهد، و دلیلش mean-centering است.** (Ana در واقع به Terminator نمره‌ی ۱ داد.)

مثال ۴ (Q4) استاد: User 1 به Item C با Jaccard و همه‌ی فیلم‌های امتیازداده). $U_C = \{2, 4\}$.

- $J(C, A) = |\{2,4\}|/|\{1,2,3,4\}| = 2/4 = 0.5$
- $J(C, B)$: $U_B = \{1,3,4\}$، اشتراک $\{4\}$، اجتماع $\{1,2,3,4\} \leftarrow 0.25$
- $J(C, D)$: $U_D = \{1,2,3\}$، اشتراک $\{2\}$، اجتماع ۴ ← 0.25
- User 1: A ۵، B ۳، D ۱ ← $\hat{r} = \dfrac{0.5 \times 5 + 0.25 \times 3 + 0.25 \times 1}{0.5 + 0.25 + 0.25} = \dfrac{3.5}{1.0} = 3.5$

نقش k

مثال ۱ (cosine خام، Terminator → Ana). با $k = 1$ فقط شبیه‌ترین فیلم یعنی Matrix (۰٫۹۸) استفاده می‌شود ← $\hat{r} = 0.98 \times 1/0.98 = 1.0$. با $k = 3 \leftarrow 3.02$ (مثال ۲ بالا). **انتخاب k جواب را عوض می‌کند.**

مثال ۲ (Task 3 استاد). با $k = 1$ فقط A (شباهت ۰٫۷۵) ← $\hat{r} = 4$. با $k = 2 \leftarrow 3.2$.

k **کوچک** (مثلاً ۳): پیش‌بینی به چند فیلم وابسته است ← پرنویز (variance زیاد). k **خیلی بزرگ:** فیلم‌های کم‌ربط وارد می‌شوند و وزنشان جمع می‌شود ← پیش‌بینی به سمت میانگین ساده‌ی کاربر می‌رود (bias زیاد). مقدارهای معمول ۲۰ تا ۱۰۰، با آزمایش روی داده‌ی validation. (کد استاد اصلاً k ندارد و از همه‌ی فیلم‌های امتیازداده استفاده می‌کند.)

۲.۷ Top-N: ساختن ردیف «چون X را دیدی»

پیش‌بینی یک امتیاز محصول نیست؛ محصول یک **لیست** است. دو راه:

راه ۱ — Predict & Sort. برای همه‌ی فیلم‌هایی که کاربر ندیده امتیاز پیش‌بینی کن، مرتب کن، N تای اول را بردار.

چرا استفاده نمی‌شود؟ برای یک کاربر و کاتالوگ یک میلیونی، یعنی یک میلیون پیش‌بینی، هر کدام صدها محاسبه‌ی similarity، و همه باید در چند میلی‌ثانیه (تا صفحه باز شود) تمام شوند. تازه همان شباهت‌ها برای هر کاربر دوباره حساب می‌شوند. خود استاد در کدش مجبور شد فقط ۵۰۰ فیلم تصادفی را امتیاز بدهد.

راه ۲ — kNN-map (کاری که آمازون می‌کند). کار را دو قسمت کن:

- **Offline (شب‌ها):** برای هر کالا، k کالای شبیه‌ترش را حساب و ذخیره کن. این جدول **kNN-map** است.
- **Online (وقتی صفحه باز می‌شود):** کالاهایی که کاربر دارد را بگیر، همسایه‌هایشان را از جدول بخوان، امتیازها را جمع بزن، بهترین‌ها را برگردان. **هیچ similarity ای موقع درخواست حساب نمی‌شود.**

مثال ۱ (kNN-map با Jaccard و پیشنهاد به Ana). با ۵ فیلم، $5 \times 4/2 = 10$ جفت داریم:

جفت	هر دو	حداقل یکی	Jaccard
Toy Story – Nemo	Ana, Cem	4	0.50
Toy Story – Matrix	Ana, Ben	4	0.50
Toy Story – Terminator	Ben, Cem	4	0.50
Toy Story – Titanic	Cem	4	0.25
Nemo – Matrix	Ana, Dee	4	0.50
Nemo – Terminator	Cem, Dee	4	0.50
Nemo – Titanic	Cem, Dee	3	0.67
Matrix – Terminator	Ben, Dee	4	0.50
Matrix – Titanic	Dee	4	0.25
Terminator – Titanic	Cem, Dee	3	0.67

Ana این‌ها را دیده: Toy Story، Nemo، Matrix. کاندیدها: Terminator و Titanic. برای هر کاندید، شباهتش با هر فیلم Ana را جمع بزن:

کاندید	از Toy Story	از Nemo	از Matrix	جمع
Terminator	0.50	0.50	0.50	**1.50**
Titanic	0.25	0.67	0.25	1.17

پیشنهاد اول به Ana: Terminator. باز اشتباه! Jaccard فقط می‌بیند که Ana فیلم Matrix را امتیاز داده، نه این‌که یک ستاره داده. فیلمی که از آن متنفر بود مثل فیلمی که انتخاب کرده حساب شد.

مثال ۲ (همان کار، فقط با سیگنال مثبت). در سیستم‌های واقعی جدول فقط از **سیگنال‌های مثبت** ساخته می‌شود: خرید، تماشای کامل، لایک، یا امتیاز ۴ به بالا. امتیاز ۱ سیگنال «با هم دیده شدن» نیست.

فیلم	کسانی که دوست داشتند (۴ به بالا)
Toy Story	{Ana, Cem}
Nemo	{Ana, Cem}
Matrix	{Ben, Dee}
Terminator	{Ben, Dee}
Titanic	{ }

حالا $J(\text{Toy Story}, \text{Nemo}) = 2/2 = 1.0$، $J(\text{Matrix}, \text{Terminator}) = 1.0$ و هر جفت انیمیشن–اکشن $0/4 = 0$. Ana فقط Toy Story و Nemo را دوست داشت، پس score Terminator $= 0 + 0 = 0$ است. **دیگر Terminator به Ana پیشنهاد نمی‌شود.**

درس: فرمول similarity مهم است، ولی این‌که **کدام تعامل را سیگنال مثبت حساب کنی** به همان اندازه مهم است.

سه روش ترکیب امتیاز همسایه‌ها

استاد سه روش نام می‌برد: بیشترین **تکرار** در C، بیشترین **جمع** شباهت، یا بیشترین **ماکسیمم** شباهت.

مثال ۱. کاربر X و Y را دیده؛ $\text{knn_map}[X] = [(0.9, P), (0.5, Q)]$ و $\text{knn_map}[Y] = [(0.6, Q), (0.4, R)]$.

قاعده	P	Q	R	ترتیب
جمع	0.9	$0.5 + 0.6 = 1.1$	0.4	Q، P، R
ماکسیمم	0.9	0.6	0.4	P، Q، R
تعداد	1	2	1	Q، سپس P یا R

«جمع» به کالاهایی پاداش می‌دهد که به **چند** چیزِ مورد علاقه‌ی کاربر وصل‌اند، که معمولاً همان چیزی است که می‌خواهیم.

مثال ۲. کاربر سه فیلم دیده با این همسایه‌ها: A: $[(0.8, S), (0.3, T)]$، B: $[(0.4, S), (0.7, U)]$، C: $[(0.5, T), (0.6, U)]$.

- جمع: $S = 1.2$، $T = 0.8$، $U = 1.3$ ← U، S، T
- ماکسیمم: $S = 0.8$، $T = 0.5$، $U = 0.7$ ← S، U، T
- تعداد: هر سه ۲ بار ← مساوی

۲.۸ نگه داشتن k بهترین: لیست مرتب محدود و Heap

برای ساختن kNN-map باید برای هر کالا، از بین هزاران کاندید، k تای بهتر را نگه داریم، بدون این‌که کل لیست را مرتب کنیم.

روش استاد (هفته‌ی ۲): لیست مرتبِ محدود. لیستی با حداکثر K عضو که صعودی مرتب است؛ `sorted_list[0]` همیشه **ضعیف‌ترینِ بهترین‌ها**ست. برای هر کاندید جدید: اگر لیست پر نیست، با `bisect.insort` درجش کن؛ اگر پر است و کاندید از `sorted_list[0]` بهتر است، درجش کن و `pop(0)` کن.

روش بهتر (هفته‌ی ۳ استاد هم اشاره می‌کند): heap **min-heap.** کمینه همیشه کوچک‌ترین عضو را در رأس دارد ($O(1)$ دسترسی). push و pop هر کدام $O(\log K)$.

چرا heap؟ پیدا کردن جای درج در لیست مرتب $O(\log K)$ است ولی خود درج و `pop(0)` در لیست $O(K)$ هستند. با heap هر قدم $O(\log K)$ است. برای c کاندید: heap $O(c \log K)$ زمان و $O(K)$ حافظه؛ مرتب کردن کامل $O(c \log c)$ زمان و $O(c)$ حافظه. با یک میلیون کاندید و $K = 30$، یعنی نگه داشتن ۳۰ عدد به جای یک میلیون.

مثال ۱ ($K = 2$). شباهت‌های Toy Story با بقیه، به این ترتیب پیمایش می‌شوند: Terminator ۰٫۶۰، Matrix ۰٫۹۰، Nemo ۰٫۹۶، Cars ۰٫۰۰، Titanic ۰٫۳۰.

قدم	کاندید	شباهت	heap قبل	کار	heap بعد (رأس = کوچک‌ترین)
1	Nemo	0.90	[ ]	کمتر از ۲ عضو ← push	$[0.90]$
2	Matrix	0.60	$[0.90]$	کمتر از ۲ عضو ← push	$[\mathbf{0.60}, 0.90]$
3	Terminator	0.30	$[0.60, 0.90]$	$0.30 < 0.60$ ← نادیده	$[\mathbf{0.60}, 0.90]$
4	Titanic	0.00	$[0.60, 0.90]$	نادیده	$[\mathbf{0.60}, 0.90]$
5	Cars	0.96	$[0.60, 0.90]$	$0.96 > 0.60$ ← pop و push	$[\mathbf{0.90}, 0.96]$

نتیجه: همسایه‌های Cars ← Toy Story (۰٫۹۶) و Nemo (۰٫۹۰).

مثال ۲ ($K = 3$). شباهت‌ها به ترتیب: $0.2,\ 0.9,\ 0.5,\ 0.1,\ 0.7,\ 0.8$.

push 0.2 ← $[0.2]$؛ push 0.9 ← $[0.2, 0.9]$؛ push 0.5 ← $[0.2, 0.5, 0.9]$؛ 0.1 کمتر از 0.2 ← نادیده؛ 0.7 بیشتر از 0.2 ← جایگزین ← $[0.5, 0.7, 0.9]$؛ 0.8 بیشتر از 0.5 ← جایگزین ← $[0.7, 0.8, 0.9]$. نتیجه: ۰٫۹، ۰٫۸، ۰٫۷.

(`heapq` پایتون min-heap است؛ برای همین مثال استاد `heapq.nsmallest(2, [(10,"a"),(8,"b"),(5,"c")])` جواب `[(5,'c'), (8,'b')]` می‌دهد.)

۲.۹ مسئله‌ی هزینه و الگوریتم آمازون

برای ساختن kNN-map همه‌ی جفت‌ها را مقایسه کردیم. با n کالا، $n(n-1)/2$ جفت داریم:

مثال ۱ (رشد $O(n^2)$).

تعداد کالا n	تعداد جفت
5	10
(MovieLens) 9,724	حدود ۴۷ میلیون
1,000,000	حدود ۵۰۰ میلیارد

ده برابر کالا ← صد برابر کار. این همان سوال Q2 استاد است.

ایده‌ی الگوریتم آمازون (Linden و همکاران، ۲۰۰۳). در یک کاتالوگ واقعی، تقریباً همه‌ی جفت‌ها **هیچ کاربر مشترکی ندارند** (کسی که قاب گوشی خریده و کسی که خاک باغچه خریده هیچ اشتراکی ندارند)، پس شباهتشان صفر است و حساب کردن صفر، کار هدر رفته است. پس به جای پیمایش همه‌ی جفت‌ها، **کاربران** را پیمایش کن و فقط جفت‌هایی را لمس کن که یک کاربر واقعاً به هم وصلشان کرده.

```
AMAZON_ITEM_TO_ITEM: for each item i: for each user u who rated i: for each other item j that u rated:
count[i][j] <- count[i][j] + 1 # i and j were rated together once more for each j that got a count:
compute sim(i, j) keep the k best
```

مثال ۲ (همسایه‌های Titanic در جدول کوچک). چه کسانی Titanic را امتیاز داده‌اند؟ Cem و Dee. Cem دیگر چه دیده؟ Toy Story، Nemo، Terminator ← هر کدام +۱. Dee دیگر چه دیده؟ Nemo، Matrix، Terminator ← هر کدام +۱.

فیلم دیگر	تعداد هم‌امتیازی با Titanic
Nemo	2
Terminator	2
Toy Story	1
Matrix	1

این شمارش دقیقاً صورت Jaccard است: $J(\text{Titanic}, \text{Nemo}) = \dfrac{2}{2+3-2} = 0.67$ ✓

مثال ۳ (صرفه‌جویی روی داده‌ی sparse). شش کالا و سه مشتری: U1 = {گوشی، قاب}، U2 = {گوشی، شارژر}، U3 = {خاک، بذر}.

- brute force: $6 \times 5/2 = 15$ جفت مقایسه می‌شوند.
- الگوریتم آمازون فقط جفت‌های واقعی را لمس می‌کند: گوشی–قاب، گوشی–شارژر، خاک–بذر ← **۳ جفت**. ۱۲ جفت دیگر شباهت صفر دارند و هرگز لمس نمی‌شوند.

در جدول کوچک ما همه به همه وصل‌اند و صرفه‌جویی صفر است (برای همین استاد می‌گوید روی MovieLens کمک زیادی نمی‌کند)، ولی در کاتالوگ میلیونی آمازون صرفه‌جویی عظیم است.

هزینه: بدترین حالت $O(n^2 m)$ (m تعداد کاربران)، ولی در عمل نزدیک $O(nm)$، چون بیشتر کاربران چند کالای کم دارند و حلقه‌ی داخلی کوتاه است.

راه‌های دیگر فرار از $O(n^2)$ (که استاد نام می‌برد): جست‌وجوی تقریبی نزدیک‌ترین همسایه مثل **LSH** (تقریباً بهترین همسایه‌ها را خیلی سریع‌تر پیدا می‌کند)، محاسبه به شکل **ضرب ماتریس روی GPU**، و **sampling** کاندیدها (کاری که کد استاد با `max_candidates=500` می‌کند).

۲.۱۰ الگوریتم‌ها و پیچیدگی

```
PREDICT(u, i, k): for each item j rated by u: s[j] <- sim(i, j) N <- the k items with the largest s[j] if sum of s over N is 0: return fallback return sum(s[j] * r[u][j] for j in N) / sum(|s[j]| for j in N) TOPN_KNN_MAP(u, N): # online C <- empty table item -> score for each item j that u interacted with: for each (sim, i) in knn_map[j]: if u has not interacted with i: C[i] <- C[i] + sim return the N items with the highest score in C
```

n کالا، m کاربر، $\bar{u}$ میانگین تعداد امتیاز هر کالا، $|I_u|$ تعداد کالاهای کاربر.

عملیات	هزینه	دلیل
یک `sim(i,j)`	$O(\bar{u})$	مقایسه‌ی دو لیست کاربر
`PREDICT(u,i)`	$O(\|I_u\| \cdot \bar{u})$	یک similarity برای هر کالای کاربر
Top-N predict & sort	$O(n \cdot \|I_u\| \cdot \bar{u})$	برای **هر** درخواست
ساخت kNN-map	زمان $O(n^2\bar{u})$، حافظه $O(nk)$	یک بار، offline
Top-N با kNN-map	$O(\|I_u\| \cdot k)$	**مستقل از اندازه‌ی کاتالوگ**
الگوریتم آمازون	بدترین $O(n^2m)$، در عمل $O(nm)$	فقط جفت‌های co-rated

ایده‌ی طراحی که در هر recommender واقعی می‌بینی: کار سنگین را offline انجام بده و قدم online را به lookup های ساده محدود کن. در صنعت کل item-kNN با **ضرب ماتریس sparse** انجام می‌شود (اگر ستون‌های ماتریس کاربر–کالا X نرمال شوند، $S = X^\top X$ همه‌ی cosine ها را می‌دهد، و ضرب sparse خودبه‌خود فقط جفت‌های co-rated را لمس می‌کند – یعنی همان الگوریتم آمازون). کتابخانه‌ها: `implicit` برای top-N روی داده‌ی implicit، `scikit-surprise` برای rating prediction، و FAISS / HNSW برای جست‌وجوی نزدیک‌ترین همسایه در مقیاس میلیونی.

۲.۱۱ کد استاد و ایرادهایش

ساختارهای داده. `rating_map["userId_movieId"]` ← امتیاز (مثلاً `rating_map["1_101"] = 5.0`)، و `user_ratings_map[movieId]` ← (لیست کاربران، لیست امتیازها). هدف: lookup با هزینه‌ی $O(1)$ به جای فیلتر کردن DataFrame در حلقه‌های تودرتو.

`calc_sim`: شاخه‌ی Jaccard مجموعه‌ی کاربران را می‌سازد و اشتراک تقسیم بر اجتماع را برمی‌گرداند؛ شاخه‌ی Cosine با `np.intersect1d(..., return_indices=True)` کاربران مشترک و جایشان در هر آرایه را پیدا می‌کند (مثال ۳ بخش ۲.۴)، و ضرب داخلی تقسیم بر دو طول را برمی‌گرداند.

`NNCF_based_rating_prediction`: همان فرمول بخش ۲.۶، روی **همه‌ی** فیلم‌های کاربر (بدون k)؛ اگر جمع شباهت صفر باشد، **صفر** برمی‌گرداند (پیش‌بینی بدی روی مقیاس ۱ تا ۵؛ بهتر است میانگین فیلم یا میانگین کل باشد).

ایرادهای کد استاد:

1. **`top_N_pred_sort`:** شرط `ratings["userId"]==1` به جای `u` نوشته شده؛ برای هر کاربری غیر از ۱، «فیلم‌های ندیده» اشتباه حساب می‌شود. و فقط ۵۰۰ فیلم تصادفی امتیاز داده می‌شود چون الگوریتم کند است.

2. **`build_knn_map`:** هر کالا فقط با ۵۰۰ کالای **تصادفی** مقایسه می‌شود (`max_candidates`)، پس همسایه‌ی واقعی اغلب اصلاً دیده نمی‌شود. `bisect.insort` و `pop(0)` هر دو $O(K)$ اند؛ heap بهتر است.
3. **نسخه‌ی آمازون:** `np.random.choice(movies, 500)` با جای‌گذاری نمونه می‌گیرد ← کاندید تکراری؛ و این sampling جزو الگوریتم آمازون نیست.
4. **`add_sims_and_sort`:** `itertools.groupby` فقط عناصر **پشت‌سرهمِ** هم‌کلید را گروه می‌کند، پس ورودی باید از قبل بر اساس movieId مرتب باشد. اینجا نیست، پس شباهت‌های یک فیلم که از چند همسایه آمده **جمع نمی‌شوند** و فیلم چند بار جدا در لیست می‌ماند. عملاً کد «ماکسیمم» حساب می‌کند نه «جمع». (مثال: لیست $[(0.9, P), (0.5, Q), (0.4, R), (0.6, Q)]$ ← کد استاد Q را دو بار جدا با ۰٫۵ و ۰٫۶ می‌دهد؛ درستش Q $= 1.1$.) اصلاح: قبل از groupby مرتب کن یا در دیکشنری جمع بزن.
5. **`top_N_knn_map`:** فیلم‌های دیده‌شده حذف نمی‌شوند.
6. **Leakage در evaluation:** `knn_map` روی **کل** `ratings` ساخته شده و بعد test جدا شده؛ پس شباهت‌ها امتیازهای test را دیده‌اند.
7. کامنت می‌گوید `ratings >= 4` ولی کد `rating > 4` است (یعنی فقط ۴٫۵ و ۵).

Hit Ratio در نوت‌بوک kNN: اینجا پروتکل با فصل ۱ فرق دارد: **بدون negative sampling**. برای هر (u, i) در test یک لیست top-N واقعی از **کل کاتالوگ** ساخته می‌شود و چک می‌شود i در آن هست یا نه. استاد با $N = 100$ عدد ۰٫۱۶۵ گرفت. این عدد با HR های فصل ۱ (رتبه بین ۱۰۰ کاندید) **قابل مقایسه نیست**.

۲.۱۲ سوال‌های امتحان — فصل ۲

سوال متن نوت‌بوک: «How can you explain the difference between using Cosine vs. Jaccard?» (در خروجی استاد Jaccard با MAE ≈ ۰٫۷۴ از Cosine با ≈ ۰٫۷۹ بهتر شد.)

1. cosine استاد فقط روی کاربران مشترک حساب می‌شود و همه‌ی امتیازها مثبت‌اند، پس تقریباً همه‌ی شباهت‌ها نزدیک ۱ می‌شوند (روی MovieLens میانگین شباهت‌های غیرصفر حدود ۰٫۹۶ است و ۹۴٪ آن‌ها بالای ۰٫۹). وقتی وزن‌ها تقریباً برابرند، میانگین وزن‌دار همان **میانگین ساده‌ی کاربر** می‌شود؛ برای همین MAE cosine تقریباً با baseline میانگین کاربر برابر است.
2. تعداد کاربران مشترک را نادیده می‌گیرد: دو فیلم با فقط **یک** کاربر مشترک cosine دقیقاً ۱ می‌گیرند — بیشترین وزن از ضعیف‌ترین شاهد.
3. Jaccard مقدار امتیاز را دور می‌ریزد ولی **قدرت co-occurrence** را اندازه می‌گیرد؛ توزیعش پهن است و واقعاً بین همسایه‌ها تمایز می‌گذارد.
4. درمان cosine: mean-centering (adjusted cosine)، محاسبه‌ی طول روی کل بردار (missing = 0)، یا significance weighting: $\text{sim}' = \frac{n_{ij}}{n_{ij} + \beta} \text{sim}$.

English: In the notebook, cosine is computed only over co-rated users and all ratings are positive, so almost every similarity is close to 1 (exactly 1 with a single common user). The weights are nearly uniform, so the prediction degenerates to the user's average. Jaccard ignores the values but captures co-occurrence strength; its values are spread out, so it discriminates between neighbours. Mean-centering and significance weighting fix cosine.

سوال بخش «Can you suggest a solution for Efficiency?» (برای این‌که امتیاز همه‌ی کالاها را پیش‌بینی نکنیم)

همه‌ی کاتالوگ را امتیاز نده. طراحی دو مرحله‌ای: (۱) **candidate generation**: چند صد کاندید را ارزان انتخاب کن، مثلاً همسایه‌های ازپیش‌محاسبه‌شده‌ی کالاهای کاربر (kNN-map) یا جست‌وجوی تقریبی نزدیک‌ترین همسایه (LSH/HNSW)؛ (۲) فقط همان‌ها را امتیاز بده و رتبه‌بندی کن. به‌علاوه، شباهت‌ها را **offline** حساب کن و فقط top-k هر کالا را نگه دار.

English: Use a two-stage design: cheap candidate generation (precomputed k nearest neighbours of the user's items, or approximate NN search) followed by scoring only those candidates; precompute item–item similarities offline and store only the top-k per item.

Q1. Jaccard و Cosine در برخورد با مقدار امتیاز چه فرقی دارند؟ کدام magnitude را نادیده می‌گیرد؟ Jaccard امتیازها را دودویی می‌بیند (امتیاز داده / نداده) و فقط **مجموعه‌ی** کاربران را مقایسه می‌کند، پس **مقدار امتیاز را نادیده می‌گیرد** (۱ ستاره و ۵ ستاره یکی‌اند). Cosine هر کالا را برداری از امتیازها می‌بیند و زاویه را اندازه می‌گیرد، پس مقدارها مهم‌اند. ::: {"en dir="ltr.

English: Jaccard treats ratings as binary and compares only the sets of users, so it ignores rating .magnitudes; cosine treats items as rating vectors and measures the angle, so values matter

Q2. پیچیدگی زمانی ساخت kNN-map کامل برای N کالا؟ چرا ناکارآمد است؟ $O(N^2)$ ← $N(N-1)/2$ جفت محاسبه‌ی similarity (هر کدام $O(\bar{u})$). رشد درجه‌ی دو برای میلیون‌ها کالا غیرممکن است (۱۰⁶ کالا ← حدود ۵×۱۰¹¹ جفت) و با تغییر داده باید دوباره ساخته شود. حافظه فقط $O(Nk)$. راه‌حل‌ها: GPU، LSH، الگوریتم آمازون، sampling.

Q3. فرق MAE و Hit Ratio؛ هر کدام کِی مناسب است؟ MAE میانگین قدرمطلق اختلاف امتیاز پیش‌بینی‌شده و واقعی است: خطای **رگرسیون** برای **rating prediction** (کمتر بهتر). Hit Ratio بررسی می‌کند کالای مرتبطِ کنارگذاشته در لیست top-N هست یا نه: معیار **رتبه‌بندی** برای **Top-N** (بیشتر بهتر). MAE امتیاز explicit لازم دارد؛ Hit Ratio فقط به بالای لیست اهمیت می‌دهد (همان چیزی که کاربر می‌بیند) و با implicit هم کار می‌کند.

Q4. Jaccard(A,D) $= 0.75$؛ Cosine(A,D) $= 0.507$؛ $\hat{r}_{U2,B} = 3.2$ (حل کامل در بخش‌های ۲.۳، ۲.۴، ۲.۶).

Q5. HR $= 3/5 = 0.60$ (حل کامل در فصل ۱، بخش ۱.۵). :::

سوال‌های محتمل دیگر:

- **الگوریتم پیش‌بینی item-based را با شبه‌کد بنویس و پیچیدگی‌اش را بده.** ← `PREDICT` بخش ۲.۱۰؛ $O(|I_u| \cdot \bar{u})$.
- **Top-N با kNN-map را بنویس؛ بخش offline و online کدام است؟** ← offline: ساخت map با $O(n^2\bar{u})$ زمان و $O(nk)$ حافظه؛ online: $O(|I_u|k)$ برای هر درخواست، مستقل از اندازه‌ی کاتالوگ.
- **Predict & Sort را با kNN-map مقایسه کن.** ← اولی برای هر کالای ندیده پیش‌بینی می‌کند و شباهت‌ها را برای هر کاربر دوباره حساب می‌کند؛ دومی شباهت‌ها را یک بار حساب می‌کند و فقط top-k را نگه می‌دارد؛ چند مرتبه‌ی بزرگی سریع‌تر، با هزینه‌ی ساخت offline.
- **الگوریتم آمازون را توضیح بده و بگو چرا روی داده‌ی sparse سریع‌تر است.** ← بخش ۲.۹.
- **چطور k بهترین را هنگام پیمایش کاندیدها کارآمد نگه داریم؟** ← heap کمینه به اندازه‌ی k؛ $O(c \log k)$ زمان، $O(k)$ حافظه.
- **چرا mean-centering برای cosine روی امتیاز explicit لازم است؟** ← بخش ۲.۴ و ۲.۵.

- **User-based در برابر item-based؛ چرا صنعت item-based را ترجیح داد؟** ← بخش ۲.۱.
- **اثر k؟** ← بخش ۲.۶.
- **محدودیت‌های kNN-CF؟** ← sparsity، cold start، هزینه‌ی $O(n^2)$، popularity bias، استفاده نکردن از محتوا.
- **آیا HR فصل ۱ (۱+۹۹) با HR نوت‌بوک kNN قابل مقایسه است؟** ← نه؛ اولی رتبه بین ۱۰۰ کاندید است و دومی در کل کاتالوگ.
- **ایراد `add_sims_and_sort` چیست؟** ← `groupby` روی لیست مرتب‌نشده (بخش ۲.۱۱).

فصل ۳ — Matrix Factorization، BPR و LightGCN (هفته‌ی ۲، بخش دوم)

۳.۱ مسئله: کاربران امتیاز نمی‌دهند، و کاتالوگ بزرگ است

سرویس بزرگ شده و دو واقعیت، روش kNN فصل ۲ را به زحمت می‌اندازد.

واقعیت ۱ — کاربران فقط تماشا می‌کنند. فقط درصد کمی از کاربران ستاره می‌دهند. چیزی که فراوان داریم **رفتار** است: پخش، کلیک، خرید، check-in. این همان **implicit feedback** فصل ۰ است. دیتاست نوت‌بوک استاد، **Gowalla**، از همین نوع است: «کاربر X در مکان Y check-in کرد».

چون فقط مثبت‌ها را داریم و بقیه **نامعلوم** است، سه چیز عوض می‌شود:

- **MAE و RMSE بی‌معنی‌اند** (امتیازی برای پیش‌بینی نیست) ← معیارهای رتبه‌بندی: Precision@K، Recall@K، NDCG@K.
- **تابع loss مخصوص رتبه‌بندی لازم است** ← BPR.
- **باید نمونه‌ی منفی بسازیم** ← negative sampling.

واقعیت ۲ — kNN سه محدودیت دارد:

1. **کاربر مشترک نداشتن یعنی شباهت صفر.** دو فیلم کم‌طرفدار که در واقع خیلی شبیه‌اند ولی هیچ‌کس هر دو را ندیده، بی‌ربط به نظر می‌رسند. در داده‌ی sparse این حالت عادی است.
2. $O(n^2)$ برای مقایسه‌ی همه‌ی کالاها، و یک جدول همسایه برای نگهداری.
3. **فقط یک قدم:** kNN می‌بیند «A و B را همان آدم‌ها دیدند». نمی‌بیند «A شبیه B است، B شبیه C است، پس A احتمالاً کمی شبیه C است».

ایده‌ی جدید: به‌جای نگه داشتن شباهت‌ها، برای هر کاربر و هر کالا یک **بردار کوتاه** (**embedding**، مثلاً ۶۴ عدد) **یاد بگیر**، طوری که بردار کاربر و بردار کالا وقتی «جور» باشند که کاربر کالا را دوست دارد. چون همه‌ی کالاها در یک فضای کوچک مشترک‌اند، دو کالایی که هرگز کاربر مشترک نداشتند هم می‌توانند نزدیک هم قرار بگیرند.

روش	کاربرد واقعی
Matrix Factorization (ALS، BPR)	اسب بارکش صنعت برای داده‌ی implicit؛ ساختن embedding برای بازیابی کاندیدها
Graph Neural Network ها (LightGCN و خانواده‌اش)	وقتی گراف «چه کسی با چه چیزی تعامل داشت» سیگنال اضافه دارد و بودجه‌ی train هست؛ PinSage در Pinterest معروف‌ترین نمونه‌ی production است

۳.۲ معیارهای رتبه‌بندی: Precision، Recall و NDCG

موقعیت. به Ana یک ردیف ۵ فیلمی نشان دادیم با این ترتیب: $[a, b, c, d, e]$. بعداً معلوم شد واقعاً این‌ها را دید (test set): $\{b, e, z\}$. ردیف چقدر خوب بود؟

جایگاه	1	2	3	4	5
فیلم	a	**b**	c	d	**e**
hit؟	خیر	بله	خیر	خیر	بله

$$\text{Precision@}K = \frac{\text{hits in top-}K}{K} \qquad \text{Recall@}K = \frac{\text{hits in top-}K}{\text{number of relevant items}}$$

- **Precision:** چه کسری از چیزهایی که نشان دادیم درست بود؟
- **Recall:** چه کسری از چیزهایی که می‌خواست را پیدا کردیم؟

چرا هر دو؟ در خلاف جهت هم می‌کشند. ۱۰۰۰ فیلم نشان بده ← recall بالا ولی precision افتضاح. یک فیلم مطمئن نشان بده ← precision بالا ولی recall پایین.

مشکل هر دو: ترتیب را نادیده می‌گیرند. لیست $[b, e, a, c, d]$ همان precision و recall لیست ما را دارد، ولی واضحاً بهتر است چون hit ها بالای لیست‌اند، همان‌جا که کاربر اول نگاه می‌کند.

NDCG — پاداش بیشتر برای hit های بالای لیست. هر hit یک «gain» می‌گیرد که با جایگاهش کم می‌شود:

جایگاه p	1	2	3	4	5
gain $= 1/\log_2(p+1)$	1.000	0.631	0.500	0.431	0.387

$$\text{DCG@}K = \sum_{p=1}^{K} \frac{rel_p}{\log_2(p+1)} \qquad \text{IDCG@}K = \sum_{p=1}^{\min(K,|\text{relevant}|)} \frac{1}{\log_2(p+1)} \qquad \text{NDCG@}K = \frac{\text{DCG}}{\text{IDCG}}$$

چرا تقسیم بر IDCG؟ IDCG همان DCG بهترین لیست ممکن است. کاربری با ۱۰ فیلم مرتبط می‌تواند gain بیشتری از کاربری با ۱ فیلم جمع کند. تقسیم بر بهترین مقدار ممکن، همه را روی مقیاس ۰ تا ۱ می‌آورد تا بشود منصفانه میانگین گرفت. «N» یعنی همین نرمال‌سازی. **چرا «K@»؟** کاربران فقط صفحه‌ی اول را نگاه می‌کنند؛ فرقی ندارد فیلم مرتبط در جایگاه ۵۰۰ باشد یا ۵۰۰۰.

مثال ۱. $\text{predicted} = [a, b, c, d, e]$، $\text{actual} = \{b, e, z\}$، $K = 5$.

- Precision@5 $= 2/5 = 0.40$؛ Recall@5 $= 2/3 = 0.67$
- DCG: hit ها در جایگاه ۲ و ۵ ← $0.631 + 0.387 = 1.018$
- IDCG: سه فیلم مرتبط ← بهترین حالت جایگاه‌های ۱، ۲، ۳ ← $1 + 0.631 + 0.5 = 2.131$
- NDCG@5 $= 1.018/2.131 = 0.478$

مثال ۲ (همان کاربر، لیست بهتر). $[b, e, a, c, d]$: Precision و Recall همان 0.40 و 0.67. ← $\text{DCG} = 1 + 0.631 = 1.631$ NDCG $= 1.631/2.131 = 0.765$. **دقت و فراخوانی یکسان، NDCG خیلی بالاتر.**

مثال ۳. کاربر ۲ فیلم مرتبط دارد و hit ها در جایگاه‌های ۲ و ۴ از top-5 هستند. DCG $= 0.631 + 0.431 = 1.062$؛ IDCG $= 1 + 0.631 = 1.631$؛ Recall@5 $= 2/2 = 1.0$. Precision@5 $= 2/5 = 0.4$، NDCG $= 0.651$.

رابطه با فصل ۱: اگر هر کاربر دقیقاً یک فیلم test داشته باشد، Recall@K همان Hit Ratio@K است.

۳.۳ Matrix Factorization: توصیف کاربر و فیلم با چند عدد

ایده. فرض کن هر فیلم را بشود با دو عدد توصیف کرد: چقدر *انیمیشن است* و چقدر *اکشن است*. و هر کاربر را با دو عدد: چقدر *انیمیشن دوست دارد* و چقدر *اکشن دوست دارد*. آن‌وقت علاقه‌ی پیش‌بینی‌شده:

$$\text{score}(u,i) = (\text{likes animation}) \times (\text{is animation}) + (\text{likes action}) \times (\text{is action})$$

این یک **ضرب داخلی** دو بردار کوتاه است:

$$\hat{x}_{ui} = \mathbf{p}_u \cdot \mathbf{q}_i = \sum_{f=1}^{d} p_{uf}\, q_{if}$$

مثال ۱ (Ana). بردارها به صورت [انیمیشن، اکشن]:

	بردار
Ana	$[2.0,\ 0.5]$
Ben	$[0.4,\ 2.1]$
Toy Story	$[2.2,\ 0.4]$
Matrix	$[0.2,\ 2.0]$
Terminator	$[0.1,\ 2.2]$

- Ana · Toy Story $= 2.0 \times 2.2 + 0.5 \times 0.4 = 4.4 + 0.2 = 4.6$ (او ۵ داده بود)
- Ana · Matrix $= 2.0 \times 0.2 + 0.5 \times 2.0 = 0.4 + 1.0 = 1.4$ (او ۱ داده بود)
- **Ana · Terminator** $= 2.0 \times 0.1 + 0.5 \times 2.2 = 0.2 + 1.1 = \mathbf{1.3}$

مدل سوال ما را درست جواب می‌دهد، و به دلیل درست: Terminator عدد «اکشن» بزرگی دارد و Ana عدد کوچکی. و این پیش‌بینی به **هیچ** کاربری که Terminator و فیلم دیگری را با هم دیده باشد احتیاج نداشت؛ فقط دو بردار لازم بود.

مثال ۲ (Ben).

- Ben · Matrix $= 0.4 \times 0.2 + 2.1 \times 2.0 = 0.08 + 4.2 = 4.28$ (او ۵ داده بود)
- Ben · Terminator $= 0.4 \times 0.1 + 2.1 \times 2.2 = 0.04 + 4.62 = 4.66$ (او ۴ داده بود)
- Ben · Toy Story $= 0.4 \times 2.2 + 2.1 \times 0.4 = 0.88 + 0.84 = 1.72$ (او ۱ داده بود)

سه نکته درباره‌ی MF:

1. **کسی این بردارها را نمی‌نویسد.** در مدل واقعی ما «انیمیشن» و «اکشن» را انتخاب نمی‌کنیم. مدل با اعداد تصادفی شروع می‌کند و آن‌ها را تنظیم می‌کند تا score ها با داده جور شوند. ابعادی که کشف می‌کند **latent factor** نام دارند؛ اغلب معنی پیدا می‌کنند (ژانر، جدی بودن، سن مخاطب) ولی نه همیشه. اندازه‌ی معمول: ۳۲ تا ۲۵۶ عدد.
2. **اسم.** همه‌ی بردارهای کاربر را سطرهای ماتریس P و همه‌ی بردارهای کالا را سطرهای Q بگذار؛ کل جدول score ها $PQ^\top$ است. ماتریس بزرگ امتیاز را با ضرب دو ماتریس باریک تقریب می‌زنیم، یعنی **تجزیه (factorize)** می‌کنیم: $R \approx PQ^\top$.

3. **فشرده بودن.** در PyTorch، `nn.Embedding(num_users, 64)` دقیقاً همان ماتریس P است: جدولی با یک سطر قابل‌آموزش برای هر کاربر.

مثال ۳ (تعداد پارامتر). جدول کوچک: $5 \times 4 = 20$ خانه؛ MF با $d = 2$: $(4 + 5) \times 2 = 18$ عدد — صرفه‌جویی ندارد. ولی ۱۰۰ میلیون کاربر × یک میلیون کالا $= 10^{14}$ خانه است؛ با $d = 64$: $(10^8 + 10^6) \times 64 \approx 6.5 \times 10^9$ عدد. این جا می‌شود؛ جدول کامل هرگز.

۳.۴ BPR: بردارها از داده‌ی implicit چطور یاد گرفته می‌شوند؟

مسئله. با امتیاز می‌شد گفت «امتیاز پیش‌بینی‌شده را به امتیاز واقعی نزدیک کن». با implicit امتیازی نیست. فقط می‌دانیم: *Ana فیلم Toy Story را دیده؛ Terminator را ندیده.*

ایده‌ی BPR (Bayesian Personalized Ranking). لازم نیست score عدد خاصی باشد؛ فقط **ترتیب** باید درست باشد:

> کالایی که کاربر با آن تعامل داشته باید score بالاتری از کالایی بگیرد که با آن تعامل نداشته.

پس هر نمونه‌ی آموزشی یک **سه‌تایی** (u, i, j) است: کاربر u، کالای مثبت i (تعامل داشته)، و کالای j که **تصادفی** انتخاب شده و تعامل نداشته (منفی). انتخاب تصادفی j همان **negative sampling** است.

$$\text{loss} = -\ln \sigma(\hat{x}_{ui} - \hat{x}_{uj}) \qquad \sigma(z) = \frac{1}{1 + e^{-z}}$$

و روی همه‌ی سه‌تایی‌ها: $L_{BPR} = -\sum_{(u,i,j)} \ln \sigma(\hat{x}_{ui} - \hat{x}_{uj}) + \lambda \|\Theta\|^2$ (جمله‌ی دوم regularization است).

فرمول را از داخل به بیرون بخوان:

1. $\hat{x}_{ui} - \hat{x}_{uj}$ **فاصله (gap)** است: مثبت چقدر از منفی جلوتر است.
2. σ این فاصله را به عددی بین ۰ و ۱ می‌برد: «مدل چقدر مطمئن است که ترتیب درست است».
3. $-\ln$ اطمینان بالا را به loss کوچک و اطمینان پایین را به loss بزرگ تبدیل می‌کند.

این یک loss **pairwise** است (دو کالا را برای یک کاربر مقایسه می‌کند)، در برابر loss **pointwise** مثل MSE که هر کالا را جدا نگاه می‌کند. برای رتبه‌بندی، pairwise طبیعی‌تر است.

مثال ۱ (جدول رفتار loss).

gap	$\sigma(\text{gap})$	loss	معنی
$+3$	0.95	0.05	ترتیب واضحاً درست ← تقریباً چیزی برای اصلاح نیست
0	0.50	0.69	نامطمئن (شروع train همین‌جاست)
-3	0.05	3.05	ترتیب واضحاً غلط ← اصلاح بزرگ

مثال ۲. $\hat{x}_{ui} = 2.0$ و $\hat{x}_{uj} = 0.5$. gap $= 1.5$؛ $\sigma(1.5) = 1/(1 + e^{-1.5}) = 1/(1 + 0.223) = 0.818$؛ loss $= -\ln 0.818 = 0.201$.

اگر جابه‌جا بودند (gap $= -1.5$): $\sigma(-1.5) = 0.182$ و loss $= -\ln 0.182 = 1.70$.

مثال ۳ (یک قدم آموزش با دست). کاربر Ana، مثبت i = Toy Story، منفی j = Terminator. بردارهای اولیه‌ی بد: $\mathbf{p}_u = [1, 0]$، $\mathbf{q}_i = [0.5, 0.5]$، $\mathbf{q}_j = [1, 1]$؛ نرخ یادگیری ۰٫۱.

1. score ها: $\hat{x}_{ui} = 1 \times 0.5 + 0 \times 0.5 = 0.5$؛ $\hat{x}_{uj} = 1 \times 1 + 0 \times 1 = 1.0$ ← منفی جلوتر است. ترتیب غلط.
2. gap $= -0.5$؛ $\sigma(-0.5) = 0.378$؛ loss $= -\ln 0.378 = 0.974$.
3. اندازه‌ی اصلاح: $g = 1 - \sigma(\text{gap}) = 0.622$ (اگر ترتیب درست بود، g نزدیک صفر می‌شد و تقریباً چیزی عوض نمی‌شد).
4. هر بردار یک قدم کوچک در جهتی می‌رود که gap را بزرگ کند:
   - کاربر: $\mathbf{p}_u \leftarrow \mathbf{p}_u + 0.1\, g\, (\mathbf{q}_i - \mathbf{q}_j) = [1, 0] + 0.0622 \times [-0.5, -0.5] = [0.969,\ -0.031]$
   - مثبت: $\mathbf{q}_i \leftarrow \mathbf{q}_i + 0.1\, g\, \mathbf{p}_u = [0.5, 0.5] + 0.0622 \times [1, 0] = [0.562,\ 0.5]$ ← **به سمت** کاربر کشیده شد
   - منفی: $\mathbf{q}_j \leftarrow \mathbf{q}_j - 0.1\, g\, \mathbf{p}_u = [1, 1] - 0.0622 \times [1, 0] = [0.938,\ 1]$ ← **از** کاربر دور شد
5. gap جدید $= -0.348$، loss جدید $= 0.882$. هنوز غلط، ولی بهتر. این را میلیون‌ها بار تکرار کن تا بردارها جا بیفتند.

«آموزش» یعنی همین: برای هر تعامل مشاهده‌شده، کاربر و کالا را به هم نزدیک کن و یک کالای تصادفی دیگر را دور کن.

```
BPR_TRAIN(interactions, d, lr, epochs): P <- small random numbers (one row of d numbers per user) Q <- small random numbers (one row of d numbers per item) repeat `epochs` times: for each observed interaction (u, i): # in practice: mini-batches j <- a random item # negative sampling gap <- P[u].Q[i] - P[u].Q[j] g <- 1 - sigmoid(gap) P[u] <- P[u] + lr * g * (Q[i] - Q[j]) Q[i] <- Q[i] + lr * g * P[u] Q[j] <- Q[j] - lr * g * P[u] RECOMMEND(u, N): scores <- Q . P[u]; remove items u already has; return top-N
```

در کد استاد این سه خط به‌روزرسانی نوشته نشده‌اند: او loss را حساب می‌کند و `loss.backward()` را صدا می‌زند؛ PyTorch همین گرادیان‌ها را خودکار حساب می‌کند و Adam اعمالشان می‌کند. کد loss در نوت‌بوک: `loss = -F.logsigmoid(pos_scores - neg_scores).mean()`.

۳.۵ LightGCN: اشتراک اطلاعات بین همسایه‌ها در گراف

تصویر دیگری از همان داده. هر کاربر و هر کالا را یک نقطه بکش، و اگر کاربر با کالا تعامل داشته، بینشان خط بکش. این یک **گراف دوبخشی (bipartite)** است: دو نوع نقطه، و خط فقط بین دو نوع. مثال: Ana فیلم‌های Toy Story و Nemo را دیده؛ Cem فیلم‌های Nemo و Titanic را.

```
Ana ---- Toy Story \ ---- Nemo / Cem ---- Titanic
```

Ana و Cem **از طریق** Nemo به هم وصل‌اند. و Ana از مسیر Ana – Nemo – Cem – Titanic به Titanic وصل است. **این مسیر همان سیگنال collaborative است:** *کسی که یک فیلم مشترک با Ana دارد، Titanic را هم دیده.*

MF با این چه می‌کند؟ بردار Ana فقط از خط‌های خود Ana به‌روز می‌شود؛ اطلاعات Cem فقط آهسته و غیرمستقیم به او می‌رسد.

LightGCN چه می‌کند؟ قبل از حساب کردن score، به هر نقطه اجازه می‌دهد **بردارش را با بردار همسایه‌هایش مخلوط کند**، و این را چند بار تکرار می‌کند. هر دور یک **layer** است و این مخلوط کردن **message passing** نام دارد.

- بعد از ۱ لایه: بردار Ana شامل فیلم‌های خودش است (Toy Story، Nemo).

- بعد از ۲ لایه: شامل کاربرانی که آن فیلم‌ها را دارند (Cem) — همان ایده‌ی «کاربران شبیه» فصل ۲.
- بعد از ۳ لایه: شامل فیلم‌های آن کاربران (Titanic) — یعنی کاندیدهای پیشنهاد.

پس LightGCN ایده‌ی همسایگی kNN را داخل مدل embedding می‌برد. این پل بین فصل ۲ و فصل ۳ است.

ریاضی LightGCN

ماتریس مجاورت کل گراف با $(m+n)$ گره:

$$A = \begin{bmatrix} 0 & R \\ R^\top & 0 \end{bmatrix}$$

قانون propagation (بدون ماتریس وزن و بدون تابع فعال‌سازی — برای همین «Light»):

$$\mathbf{e}_u^{(k+1)} = \sum_{i \in N_u} \frac{1}{\sqrt{|N_u|}\sqrt{|N_i|}} \mathbf{e}_i^{(k)} \qquad \Longleftrightarrow \qquad E^{(k+1)} = \hat{A}\, E^{(k)}, \quad \hat{A} = D^{-1/2} A D^{-1/2}$$

بردار نهایی = میانگین همه‌ی لایه‌ها (از جمله لایه‌ی ۰)، و score مثل MF:

$$\mathbf{e}_u = \frac{1}{K+1} \sum_{k=0}^{K} \mathbf{e}_u^{(k)} \qquad \hat{x}_{ui} = \mathbf{e}_u^\top \mathbf{e}_i$$

چرا normalization ($D^{-1/2}AD^{-1/2}$)؟ بدون آن، گره‌ای با ۱۰٬۰۰۰ همسایه ۱۰٬۰۰۰ بردار را جمع می‌زند و بزرگ می‌شود، در حالی که گره‌ای با ۲ همسایه کوچک می‌ماند. train ناپایدار می‌شود و کالاهای محبوب همه چیز را می‌گیرند. ضریب $\frac{1}{\sqrt{\deg(i)\deg(j)}}$ پیام روی هر خط را با درجه‌ی **هر دو** سر خط مقیاس می‌کند. «Symmetric» چون گراف undirected است. این همان `d_inv` در کد استاد است ($D^{-1/2}$).

مثال ۱ (گراف ۴ گره‌ای استاد). گره ۰ به ۱، ۲، ۳ وصل است؛ ۱ به ۰ و ۲؛ ۲ به ۰ و ۱؛ ۳ فقط به ۰. درجه‌ها: $[3, 2, 2, 1]$.

- خط (۰،۱): $1/\sqrt{3 \times 2} = 0.408$ — hub با درجه‌ی ۳ ← بیشترین جریمه
- خط (۱،۲): $1/\sqrt{2 \times 2} = 0.500$
- خط (۰،۳): $1/\sqrt{3 \times 1} = 0.577$ — گره ۳ کاملاً به ۰ وابسته است ← جریمه‌ی کمتر

$d_{inv} = [1/\sqrt{3},\ 1/\sqrt{2},\ 1/\sqrt{2},\ 1] = [0.577,\ 0.707,\ 0.707,\ 1]$.

مثال ۲ (گراف Ana و Cem). $R = \begin{bmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \end{bmatrix}$ (سطرها Ana، Cem؛ ستون‌ها Toy Story، Nemo، Titanic). درجه‌ها: Ana ۲، Nemo ۲، Titanic ۱، Toy Story ۱، Cem ۲.

خط	محاسبه	وزن
Ana – Toy Story	$1/\sqrt{2 \times 1}$	0.707
Ana – Nemo	$1/\sqrt{2 \times 2}$	0.500
Cem – Nemo	$1/\sqrt{2 \times 2}$	0.500
Cem – Titanic	$1/\sqrt{2 \times 1}$	0.707

مثال ۳ (message passing بدون normalization، مثال torch استاد). گره ۰ به ۱ و ۲ وصل است. بردارها: گره ۰ $= [10, 10]$، گره ۱ $= [1, 2]$، گره ۲ $= [3, 4]$. با `torch.sparse.mm(A, E)`:

- گره ۰ = گره ۱ + گره ۲ $= [1+3,\ 2+4] = [4, 6]$
- گره ۱ = گره ۰ $= [10, 10]$؛ گره ۲ = گره ۰ $= [10, 10]$

ضرب ماتریس مجاورت در ماتریس embedding یعنی «هر گره = جمع همسایه‌هایش». با normalization این جمع به میانگین وزن‌دار تبدیل می‌شود.

مثال ۴ (propagation کامل روی گراف Ana و Cem). بردارهای لایه‌ی ۰: Nemo $[1, 1]$، Toy Story $[1, 1]$، Cem $[0, 1]$، Ana $[1, 0]$، Titanic $[2, 0]$ $[0, 2]$.

لایه‌ی ۱ (هر گره = جمع وزن‌دار همسایه‌ها):

- Ana $= 0.707 \times [1,1] + 0.5 \times [2,0] = [1.707,\ 0.707]$
- Cem $= 0.5 \times [2,0] + 0.707 \times [0,2] = [1.0,\ 1.414]$
- Toy Story $= 0.707 \times \text{Ana} = [0.707,\ 0]$
- Nemo $= 0.5 \times \text{Ana} + 0.5 \times \text{Cem} = [0.5,\ 0.5]$
- Titanic $= 0.707 \times \text{Cem} = [0,\ 0.707]$

لایه‌ی ۲ (همان قانون روی بردارهای لایه‌ی ۱):

- Ana $= 0.707 \times [0.707, 0] + 0.5 \times [0.5, 0.5] = [0.75,\ 0.25]$
- Titanic $= 0.707 \times [1.0, 1.414] = [0.707,\ 1.0]$

بردار نهایی = میانگین لایه‌های ۰، ۱، ۲:

- Ana $= ([1,0] + [1.707, 0.707] + [0.75, 0.25])/3 = [1.152,\ 0.319]$
- Titanic $= ([0,2] + [0, 0.707] + [0.707, 1.0])/3 = [0.236,\ 1.236]$

score قبل و بعد:

- MF ساده با بردارهای اولیه: Ana · Titanic $= 1 \times 0 + 0 \times 2 =$ **۰**
- LightGCN: Ana · Titanic $= 1.152 \times 0.236 + 0.319 \times 1.236 = 0.272 + 0.394 =$ **۰٫۶۷**

Ana هرگز Titanic را لمس نکرده و بردارهای اولیه می‌گفتند «هیچ ارتباطی نیست». بعد از دو دور مخلوط کردن، گراف اطلاعات Cem را به Ana رسانده و Titanic score مثبت گرفته است.

چرا میانگین لایه‌ها و نه فقط لایه‌ی آخر؟

1. **Over-smoothing:** اگر مدام مخلوط کنی، بردار هر گره ترکیبی از همه می‌شود و همه‌ی بردارها شبیه هم می‌شوند؛ مدل دیگر نمی‌تواند کاربرها را از هم جدا کند. نگه داشتن لایه‌های اولیه در میانگین، هویت هر گره را حفظ می‌کند.
2. لایه‌های مختلف اطلاعات متفاوت دارند (خود گره، کالاهایش، کاربران شبیه، کالاهای آن‌ها)؛ میانگین همه را نگه می‌دارد.

چرا «Light»؟ GNN های معمولی در هر لایه در یک ماتریس وزن قابل‌آموزش ضرب می‌کنند و تابع غیرخطی می‌زنند. مقاله‌ی LightGCN نشان داد برای recommendation این دو قدم کمکی نمی‌کنند و train را سخت‌تر می‌کنند، پس حذفشان کرد.

رابطه با MF: پارامترهای قابل‌آموزش **دقیقاً یکی است** (بردار لایه‌ی ۰ برای هر کاربر و کالا؛ مخلوط کردن پارامتر ندارد). loss هم یکی است (BPR). **با ۰ لایه، LightGCN دقیقاً همان MF است.**

```
LIGHTGCN_FORWARD(E0, A_hat, K): # E0: the ONLY trainable numbers E <- E0 ; layers <- [E0] for k = 1..K:
E <- A_hat @ E # one round of message passing = one sparse product layers.append(E) E_final <-
mean(layers) split E_final into user vectors and item vectors score(u, i) <- e_u . e_i # then train
with BPR exactly like MF
```

هزینه: هر دور هر خط گراف را یک بار لمس می‌کند: $O(|E| \cdot d)$؛ K لایه ← K برابر. MF فقط بردارهای batch فعلی را لازم دارد. برای همین در خروجی استاد هر epoch LightGCN کندتر از MF است.

Graph directed یا undirected؟ (یادداشت نوت‌بوک) هر دو ممکن است. گراف کاربر–کالا اینجا undirected است (اگر کاربر با کالا تعامل داشته، یال در هر دو جهت اضافه می‌شود)، برای همین normalization متقارن است. گراف directed مثل «A، B را follow کرده» در توییتر، message passing مخصوص لازم دارد.

۳.۶ کد استاد (`gnn.ipynb`) و ایرادهایش

Pipeline: (۱) Gowalla ← فقط ستون‌های (user, item) ← فیلتر **core-50** (کاربر و کالای با حداقل ۵۰ تعامل، برای سرعت) ← تبدیل ID ها به اندیس پیوسته‌ی `0..N-1` (لازم برای `nn.Embedding`)؛ (۲) split تصادفی ۸۰/۲۰ برای هر کاربر؛ (۳) ماتریس sparse دودویی `R`؛ (۴) Precision/Recall/NDCG و `evaluate_model` که embedding ها را **یک بار** می‌گیرد، برای هر کاربر `u_emb @ items_emb.T` حساب می‌کند، کالاهای train را `-inf` می‌کند و `torch.topk` می‌گیرد؛ (۵) `MatrixFactorization` با دو `nn.Embedding`؛ (۶) `LightGCN` با `_build_norm_adj` و حلقه‌ی `torch.sparse.mm` و `torch.mean` روی لایه‌ها؛ (۷) `BPRDataset` با `neg_item = randint(num_items)`؛ (۸) Adam و BPR loss.

ایرادها:

1. **Leakage و تکرار:** Gowalla check-in است و کاربر می‌تواند چند بار به یک مکان برود. split روی check-in ها انجام شده نه روی جفت‌های یکتا؛ در خروجی استاد هم پیداست (`User 0: [32, 27, 1, 1, 46]`). پس یک جفت (user, item) می‌تواند هم در train باشد هم در test؛ و چون کالاهای train در evaluation با `-inf` حذف می‌شوند، آن کالاهای test **هرگز** hit نمی‌شوند ← recall مصنوعی پایین. اصلاح: `drop_duplicates` قبل از split.
2. **Negative sampling ساده:** `randint` چک نمی‌کند کالا واقعاً منفی باشد (false negative)؛ تقریب رایج و ارزان چون داده خیلی sparse است.
3. **بدون L2 regularization** — در مقاله‌های BPR و LightGCN هست.
4. **منطق propagation دو بار کپی شده** (`forward` و `get_embeddings`)؛ اسم `g_droped_adj` باقی مانده ولی dropout ای نیست.
5. ساختن adjacency با `dok_matrix` و حلقه کند است.
6. **فقط ۱۰ epoch:** هر دو مدل کم‌آموزش‌اند (Recall@20 برای MF حدود ۰٫۰۹)، پس مقایسه‌ی نهایی دقیق نیست.
7. split تصادفی؛ در عمل باید زمانی باشد (آینده از روی گذشته؛ فصل ۴).

نتیجه روی MovieLens (امتیاز ۴+ = مثبت، ۸۰/۲۰، Recall@20): popularity ۰٫۱۴۵؛ MF+BPR ۰٫۱۸۶؛ item-kNN ۰٫۲۰۵؛ LightGCN ۰٫۲۲۵. با پارامترها و loss یکسان، فقط اضافه کردن message passing به MF عدد را بالا برد. item-kNN ساده

هم کاملاً رقابتی بود؛ برای همین در صنعت همیشه baseline های ساده‌ی خوب‌تنظیم‌شده را کنار مدل deep نگه می‌دارند.

ابزارها: `implicit` (ALS و BPR)، PyTorch Geometric (`LightGCN` آماده)، RecBole، و FAISS برای serving.

۳.۷ سوال‌های امتحان — فصل ۳

در `gnn.ipynb` سوال تمرینی نیست؛ این‌ها شکل‌های محتمل سوال‌اند.

C1. فرق explicit و implicit و اثرش بر ارزیابی؟ explicit ترجیح اعلام‌شده است (امتیاز)؛ implicit رفتار مشاهده‌شده است (کلیک، check-in، خرید): فقط مثبت‌ها دیده می‌شوند و خانه‌ی خالی نامعلوم است نه منفی. پس معیارهای خطای امتیاز بی‌معنی‌اند، از معیارهای رتبه‌بندی استفاده می‌شود و train به negative sampling احتیاج دارد.

C2 (محاسبه). NDCG@5 ، $\text{P@5} = 0.4$ ، $\text{R@5} = 0.667$ ، $\text{actual} = \{b, e, z\}$ ، $\text{predicted} = [a, b, c, d, e]$ ← $= 0.478$ (بخش ۳.۲).

C3. چرا علاوه بر Precision/Recall از NDCG استفاده می‌کنیم؟ آن دو ترتیب داخل top-K را نادیده می‌گیرند؛ NDCG هر hit را با $1/\log_2(\text{position}+1)$ تخفیف می‌دهد، پس hit های بالای لیست بیشتر حساب می‌شوند؛ تقسیم بر IDCG آن را بین ۰ و ۱ می‌برد.

C4. Matrix Factorization را توضیح بده. هر کاربر و کالا با یک بردار d بعدی؛ ترجیح پیش‌بینی‌شده ضرب داخلی آن‌هاست؛ بردارها با کمینه کردن loss روی تعامل‌های مشاهده‌شده یاد گرفته می‌شوند؛ ماتریس $m \times n$ را به $(m+n)d$ پارامتر فشرده می‌کند و به جفت‌های بدون co-occurrence هم تعمیم می‌دهد.

C5. BPR loss را بنویس و شهودش را بگو. نمونه‌ی آموزشی چیست؟ $L = -\sum \ln \sigma(\hat{x}_{ui} - \hat{x}_{uj})$؛ نمونه یک سه‌تایی (کاربر، کالای مثبت، کالای منفیِ نمونه‌گیری‌شده) است؛ loss وقتی کوچک است که مثبت بالاتر از منفی باشد، پس مدل **ترتیب نسبی** را یاد می‌گیرد نه score مطلق.

C6 (محاسبه). $\hat{x}_{ui} = 2.0$ ، $\hat{x}_{uj} = 0.5$ ← $\text{loss} = 0.201$ (جابه‌جا: 1.70).

C7. قانون propagation در LightGCN؛ فرقش با MF و GCN معمولی؟ $E^{(k+1)} = D^{-1/2} A D^{-1/2} E^{(k)}$ روی گراف دوبخشی؛ بردار نهایی میانگین لایه‌ها؛ score ضرب داخلی با BPR. در برابر GCN: بدون ماتریس وزن و بدون غیرخطی. در برابر MF: همان پارامترها، ولی embedding ها روی گراف smooth می‌شوند تا سیگنال‌های چندقدمی وارد شوند؛ با ۰ لایه همان MF.

C8. چرا normalization متقارن؟ وزن یال بین درجه‌ی ۳ و ۲؟ بدون آن گره‌های پردرجه embedding خیلی بزرگ می‌گیرند و ناپایداری عددی پیش می‌آید؛ $1/\sqrt{6} = 0.408$.

C9. بعد از ۱، ۲، ۳ لایه بردار کاربر شامل چیست؟ ۱: کالاهای خودش؛ ۲: کاربرانی که آن کالاها را دارند؛ ۳: کالاهای آن کاربران.

C10. Over-smoothing چیست و LightGCN چطور کمش می‌کند؟ با لایه‌های زیاد، میانگین‌گیری مکرر همه‌ی بردارها را شبیه هم می‌کند؛ LightGCN لایه‌ی کم استفاده می‌کند و میانگین همه‌ی لایه‌ها (شامل لایه‌ی ۰) را می‌گیرد.

C11. چرا کالاهای train موقع evaluation با $-\infty$ حذف می‌شوند؟ مدل آموزش دیده که به آن‌ها score بالا بدهد و top-K را پر می‌کنند، ولی پیشنهاد چیزی که کاربر دارد بی‌فایده است و در test هم نیستند.

C12. چرا negative sampling لازم است و ضعفش در نوت‌بوک چیست؟ داده‌ی implicit فقط مثبت دارد؛ BPR یک کالای «کمتر ترجیح‌داده‌شده» لازم دارد. نوت‌بوک کالای تصادفی را بدون چک تاریخچه‌ی کاربر انتخاب می‌کند (false negative)، و منفی‌های uniform بیشتر آسان‌اند.

C13. برای ماتریس $R = \begin{bmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \end{bmatrix}$ ماتریس A و درجه‌ها را بده. ماتریس A 5×5 است؛ درجه‌ها: $u_0 = 2,\ u_1 = 2,\ i_0 = 1,\ i_1 = 2,\ i_2 = 1$؛ مثلاً $\hat{A}_{u_0,i_0} = 1/\sqrt{2} = 0.707$ و $\hat{A}_{u_0,i_1} = 1/\sqrt{4} = 0.5$.

فصل ۴ — Content-based Recommendation (هفته‌ی ۳)

۴.۱ مسئله: فیلم جدید، صفر امتیاز

جمعه سرویس یک فیلم جدید منتشر می‌کند: **Cars**، یک انیمیشن خانوادگی. صبح جمعه **صفر امتیاز و صفر بازدید** دارد. مدیر محصول: *«Cars را در صفحه‌ی اول کسانی بگذار که دوستش خواهند داشت.»*

با روش‌های فصل‌های قبل امتحان کن:

- **item-kNN CF** کاربرانی لازم دارد که Cars و چیز دیگری را با هم دیده باشند. هیچ‌کس نیست ← همه‌ی شباهت‌های Cars صفر.
- **MF / LightGCN** بردار Cars را فقط از تعامل‌هایش یاد می‌گیرند. تعاملی نیست ← بردارش هنوز تصادفی است.
- **Popularity** به Cars عدد صفر می‌دهد ← ته لیست.

همه‌ی روش‌های CF برای کالای جدید کورند. این مسئله‌ی **item cold-start** است و دلیل اصلی وجود content-based recommendation.

ولی یک انسان فوراً جواب را می‌بیند: *Cars انیمیشن خانوادگی است؛ Ana عاشق انیمیشن خانوادگی است؛ به Ana نشانش بده.* انسان از رفتار دیگران استفاده نمی‌کند؛ از **این‌که فیلم چیست**، یعنی **محتوای** آن، استفاده می‌کند.

تعریف. یک recommender مبتنی بر محتوا کالاهایی را پیشنهاد می‌دهد که **محتوایشان** (ویژگی‌ها، متن، تصویر) شبیه محتوای کالاهایی است که کاربر قبلاً دوست داشته. از این‌ها استفاده می‌کند:

- **محتوای کالا:** ژانر، tag، عنوان، خلاصه‌ی داستان، کارگردان، بازیگران، سال، قیمت، برند، متن یک مقاله، صدای یک آهنگ، عکس یک محصول.
- **تاریخچه‌ی خود کاربر:** همین کاربر چه چیزهایی را دوست داشته.

به کاربران دیگر احتیاج ندارد. این تفاوت کلیدی با CF است. (تعریف استاد: محتوای فیلم مثل خلاصه، کارگردان، بازیگران، ژانر، تاریخ انتشار؛ و محتوای کاربر مثل فیلم‌های دیده‌شده، سن، جنسیت.)

جدول ژانرها (شامل فیلم جدید):

فیلم	ژانرها
Toy Story	Animation, Children, Comedy
Nemo	Animation, Children, Adventure
Matrix	Action, Sci-Fi
Terminator	Action, Sci-Fi, Thriller
Titanic	Drama, Romance
Cars (جدید، بدون امتیاز)	Animation, Children, Comedy

جاهایی که content-based در محصولات واقعی ضروری است:

موقعیت	چرا CF نمی‌تواند	از چه استفاده می‌شود
کالای جدید	هنوز تعاملی ندارد	محتوا از ثانیه‌ی اول معلوم است
اخبار و کالاهای کوتاه‌عمر	خبر چند ساعت مهم است؛ تا کلیک کافی جمع شود، قدیمی شده	متن و موضوع خبر
کاریابی، املاک، دوست‌یابی	هر کالا یکتاست و سریع تمام می‌شود؛ هیچ‌وقت تعامل کافی ندارد	تطبیق ویژگی‌ها: مهارت، مکان، قیمت
long tail	کالای کم‌طرفدار تعامل کمی دارد و CF به‌ندرت پیشنهادش می‌دهد	محتوا آن را با سلیقه‌ی کاربر جور می‌کند
توضیح	«۵۰۰ کاربر شبیه تو دوستش داشتند» مبهم است	«چون مثل Toy Story یک کمدی انیمیشن است»
درخواست‌های شبه‌جست‌وجو («یک کمدی سیاه سرقتی»)	CF مفهوم کلمه و موضوع ندارد	text embedding

محدودیت‌های content-based (برای امتحان حفظ کن):

1. **مفهوم کیفیت ندارد.** یک انیمیشن افتضاح و یک شاهکار با ژانرهای یکسان، یکسان دیده می‌شوند. CF می‌داند مردم کدام را واقعاً دوست داشتند.
2. **Over-specialization (حباب فیلتر):** فقط «بیشتر از همان» پیشنهاد می‌دهد؛ کاربری که سه انیمیشن دیده فقط انیمیشن می‌گیرد؛ کمتر غافلگیرکننده است (**serendipity** پایین).
3. **ویژگی خوب لازم دارد.** اگر توصیف محتوا ضعیف باشد (فقط ژانر)، پیشنهادها ضعیف‌اند. ساختن ویژگی خوب اغلب بیشترِ کار است.
4. **cold start کاربر را حل نمی‌کند.** کاربر کاملاً جدید تاریخچه ندارد؛ چیزی برای مقایسه نیست (CF هم همین مشکل را دارد). **content-based مسئله‌ی cold start *کالا* را حل می‌کند، نه کاربر.**

۴.۲ نمایش کالا

برای مقایسه‌ی دو کالا بر اساس محتوا، اول باید هر کالا را به چیزی قابل محاسبه تبدیل کرد. دو خانواده، هر کدام با similarity خودش:

نمایش	مثال برای Toy Story	similarity
مجموعه‌ی برچسب‌ها	{Animation, Children, Comedy}	Jaccard
بردار عددی (one-hot ژانر، TF-IDF متن، embedding)	$[0.9,\ 0.2,\ \ldots]$	Cosine

استاد از هر کدام یکی استفاده می‌کند: **مجموعه‌ی ژانرها با Jaccard** و **text embedding با cosine**.

۴.۳ شباهت ژانر با Jaccard

همان فرمول فصل ۲، ولی مجموعه‌ها حالا **ژانرها** هستند، نه کاربران:

$$J(i,j) = \frac{|\text{genres}_i \cap \text{genres}_j|}{|\text{genres}_i \cup \text{genres}_j|}$$

تفاوت مهم با فصل ۲. در فصل ۲ مجموعه‌ها «کاربرانی که امتیاز داده‌اند» بودند، پس شباهت از رفتار می‌آمد و هر روز عوض می‌شد. اینجا مجموعه‌ها از توصیف کالا می‌آیند، پس شباهت **قبل از این‌که کسی فیلم را ببیند** معلوم است و هیچ‌وقت عوض نمی‌شود. برای همین kNN-map محتوایی یک بار ساخته می‌شود و فقط با آمدن کالای جدید گسترش می‌یابد.

مثال ۱ (Cars با فیلم‌های Ana).

- Cars و Toy Story: مشترک $\{\text{Animation, Children, Comedy}\}$ ← ۳؛ همه ۳ ← $J = 3/3 = 1.00$
- Cars و Nemo: مشترک $\{\text{Animation, Children}\}$ ← ۲؛ همه $\{\text{Animation, Children, Comedy, Adventure}\}$ ← ۴ ← $J = 0.50$
- Cars و Matrix: مشترک ۰ ← $J = 0$

مثال ۲ (مثال خود استاد: کاربر ۱ و From Dusk Till Dawn). FDTD = {Action, Comedy, Horror, Thriller}. میان‌بر: $|A \cup B| = |A| + |B| - |A \cap B|$.

فیلم	ژانرها	امتیاز کاربر ۱	مشترک با FDTD	اجتماع	Jaccard
Toy Story	Adventure, Animation, Children, Comedy, Fantasy	4	Comedy ← 1	$5+4-1=8$	$1/8 = 0.125$
Grumpier Old Men	Comedy, Romance	4	Comedy ← 1	$2+4-1=5$	$1/5 = 0.200$
Heat	Action, Crime, Thriller	4	Action, Thriller ← 2	$3+4-2=5$	$2/5 = 0.400$
Seven	Mystery, Thriller	5	Thriller ← 1	$2+4-1=5$	$1/5 = 0.200$
Usual Suspects	Crime, Mystery, Thriller	5	Thriller ← 1	$3+4-1=6$	$1/6 = 0.167$

۴.۴ پیش‌بینی امتیاز: همان میانگین وزن‌دار

$$\hat{r}_{ui} = \frac{\sum_{j \in I_u} \text{sim}(i,j) \cdot r_{uj}}{\sum_{j \in I_u} \text{sim}(i,j)}$$

فقط similarity عوض شده. منطق: *اگر فیلم جدید از نظر محتوا خیلی شبیه فیلمی است که کاربر امتیاز داده، آن امتیاز نظرش را درباره‌ی فیلم جدید نشان می‌دهد.*

مثال ۱ (Ana → Cars، کالای cold-start).

فیلمی که Ana امتیاز داده	امتیاز	شباهت با Cars	شباهت × امتیاز
Toy Story	5	1.00	5.00
Nemo	4	0.50	2.00
Matrix	1	0.00	0.00
جمع		**1.50**	**7.00**

$$\hat{r}_{\text{Ana,Cars}} = 7.00/1.50 = 4.67$$

مثال ۲ (Ben → Cars). Ben: Toy Story ۱، Matrix ۵، Terminator ۴. شباهت‌ها با Cars: ۱، ۰، ۰.

$$\hat{r}_{\text{Ben,Cars}} = \frac{1\times 1 + 0\times 5 + 0\times 4}{1+0+0} = 1.00$$

پس: Cars را به Ana نشان بده، نه به Ben. **جواب درست، روی فیلمی با صفر امتیاز.** هیچ روش CF این کار را نمی‌توانست.

مثال ۳ (سوال همیشگی ما: Terminator → Ana با محتوا).

- Terminator و Toy Story: $0/6 = 0$؛ Terminator و Nemo: $0/6 = 0$
- Terminator و Matrix: مشترک $\{\text{Action, Sci-Fi}\}$ ← ۲؛ همه ۳ ← 0.67

$$\hat{r} = \frac{0\times 5 + 0\times 4 + 0.67\times 1}{0+0+0.67} = 1.00$$ ← دوباره درست، بدون نگاه کردن به هیچ کاربر دیگری.

ولی ضعف را ببین: پیش‌بینی فقط از **یک** فیلم (Matrix) ساخته شد. و اگر یک انیمیشن جدید افتضاح بود، این روش باز برای Ana ۴٫۶۷ پیش‌بینی می‌کرد. ***محتوا می‌گوید فیلم چیست، نه این‌که چقدر خوب است.***

مثال ۴ (ادامه‌ی مثال استاد، FDTD).

- صورت: $0.125\times 4 + 0.2\times 4 + 0.4\times 4 + 0.2\times 5 + 0.167\times 5 = 0.5 + 0.8 + 1.6 + 1.0 + 0.833 = 4.733$
- مخرج: $0.125 + 0.2 + 0.4 + 0.2 + 0.167 = 1.092$
- $\hat{r} = 4.733/1.092 = 4.34$ (استاد: ۴٫۳۳۶). امتیاز واقعی **۳** بود.

چرا ۱٫۳۴ ستاره زیادی؟ کاربر ۱ به تقریباً همه چیز ۴ و ۵ داده، و همه‌ی شباهت‌ها کوچک و نزدیک به هم‌اند (۰٫۱۳ تا ۰٫۴۰)، پس میانگین وزن‌دار نزدیک میانگین خود کاربر می‌ماند. هم‌پوشانی ژانر می‌گوید «این فیلم‌ها یک برچسب Thriller مشترک دارند»، نه «این کاربر از این یکی لذت می‌برد».

مثال ۵. کاربری A {Comedy, Romance} را ۵، B {Action, Thriller} را ۲، و C {Comedy, Drama} را ۴ داده. پیش‌بینی برای X {Comedy, Romance, Drama}:

- $J(X,A) = 2/3 = 0.667$ (مشترک Comedy، Romance؛ همه Comedy، Romance، Drama)
- $J(X,B) = 0/5 = 0$
- $J(X,C) = 2/3 = 0.667$
- $\hat{r} = \frac{0.667\times 5 + 0 + 0.667\times 4}{0.667+0+0.667} = \frac{6.0}{1.333} = 4.5$

۴.۵ بردار و cosine: one-hot، TF-IDF و embedding

Jaccard روی مجموعه‌ی ژانرها دو مشکل دارد:

1. **خشن (coarse) است.** MovieLens فقط حدود ۲۰ ژانر دارد، پس هزاران فیلم دقیقاً همان مجموعه‌ی ژانر را دارند و با هم شباهت ۱٫۰۰ می‌گیرند. خروجی خود استاد `knn_map[3]` این را نشان می‌دهد: هر ۳۰ همسایه شباهت ۱٫۰ دارند و لیست همسایه یک انتخاب دلبخواهی بین tie هاست.
2. **فقط کلمه‌ی دقیقاً یکسان را می‌شناسد.** «Animation» و «Cartoon» کاملاً متفاوت حساب می‌شوند.

راه‌حل: هر کالا را با یک **بردار** توصیف کن و با **cosine** مقایسه کن:

$$\cos(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \, \|\mathbf{b}\|}$$

(الف) One-hot ژانر. برای هر ژانر یک خانه، اگر فیلم آن ژانر را دارد ۱. همان اطلاعات مجموعه، به شکل بردار (`MultiLabelBinarizer` در sklearn؛ استاد در `build_knn_map` از آن استفاده می‌کند).

(ب) TF-IDF متن. از کلمه‌های tag، خلاصه، عنوان. هر کلمه وزن می‌گیرد:

$$\text{tf-idf}(w, d) = \underbrace{\text{tf}(w, d)}_{\text{how often in item } d} \times \underbrace{\log \frac{N}{\text{df}(w)}}_{\text{how rare among } N \text{ items}}$$

چرا IDF؟ کلمه‌ای که در همه‌ی کالاها هست («movie»، «film») هیچ چیزی درباره‌ی هیچ‌کدام نمی‌گوید و باید وزن صفر بگیرد؛ کلمه‌ی نادر («pixar»، «cyberpunk») خیلی چیز می‌گوید. TF-IDF این را خودکار انجام می‌دهد، ارزان و توضیح‌پذیر است، و موتورهای جست‌وجو روی آن ساخته شدند.

مثال ۱ (TF-IDF). سه فیلم با این tag ها: Toy Story: {movie, pixar, toys}؛ Cars: {movie, pixar, racing}؛ Terminator: {movie, robots, future}.

کلمه	در چند کالا (df)	IDF $= \ln(3/\text{df})$
movie	3	$\ln 1 = 0$
pixar	2	$\ln 1.5 = 0.41$
toys, racing, robots, future	1	$\ln 3 = 1.10$

- **بدون IDF** (شمارش ساده)، Toy Story و Terminator فقط «movie» مشترک دارند ← $\cos = 1/3 = 0.33$. فقط به خاطر یک کلمه‌ی بی‌معنی مرتبط به نظر می‌رسند.
- **با IDF**، «movie» وزن صفر می‌گیرد ← Toy Story و Terminator $= 0$. Toy Story و Cars هنوز «pixar» مشترک دارند ← $\cos = \dfrac{0.41^2}{0.41^2 + 1.10^2} = \dfrac{0.16}{1.37} = 0.12$.

مثال ۲ (TF-IDF). d1 = {cat, dog}، d2 = {cat, fish}، d3 = {cat, bird}. کلمه‌ی «cat» در هر ۳ سند هست ← $\ln(3/3) = 0$. بقیه ← $\ln(3/1) = 1.10$. پس d1 و d2 با TF-IDF شباهت صفر دارند (تنها کلمه‌ی مشترکشان همه‌جا هست)، در حالی که با شمارش ساده $\cos = 1/2 = 0.5$.

(ج) Text embedding (روش دوم استاد). یک شبکه‌ی عصبی ازپیش‌آموزش‌دیده (sentence transformer مثل `all-MiniLM-L6-v2`) متن کالا را می‌خواند و یک بردار متراکم می‌دهد (۳۸۴ عدد برای این مدل). آموزش دیده که **متن‌های هم‌معنی، بردارهای هم‌جهت بگیرند**، حتی با کلمه‌های متفاوت («animation» ≈ «cartoon»).

بردار ۳۸۴ عددی را چطور بخوانیم؟ (توضیح استاد) (۱) هر بعد یک ویژگی «پنهان» است که شبکه یاد گرفته؛ ابعاد تک‌تک معنی قابل‌خواندن ندارند. (۲) معنی در **همه‌ی** ۳۸۴ عدد پخش است؛ از یک عدد (مثلاً ۰٫۰۷۰۵-) نمی‌شود چیزی فهمید. (۳) مهم‌ترین خاصیت **موقعیت نسبی** است: متن‌های شبیه نزدیک هم قرار می‌گیرند و این نزدیکی را با cosine اندازه می‌گیریم. استاد متن هر فیلم را «عنوان + ژانرها + tag ها» می‌سازد، مثلاً: `Toy Story (1995) Adventure Animation Children Comedy Fantasy pixar pixar fun`.

مثال ۳ (cosine با بردارهای ۳ عددی اسباب‌بازی). فرض کن embedding فقط ۳ عدد داشت: [خانوادگی، اکشن، عاشقانه].

فیلم	بردار
Toy Story	$[3,1,0]$
Nemo	$[3,0,1]$
Matrix	$[1,3,0]$
Terminator	$[0,3,1]$
Titanic	$[0,0,3]$
Cars	$[3,2,0]$

- Toy Story و Cars: ضرب داخلی $3\times3+1\times2+0=11$؛ طول‌ها $\sqrt{10}=3.16$ و $\sqrt{13}=3.61$ ← $\cos = 11/11.40 = 0.96$
- Toy Story و Nemo: $9/(3.16\times3.16)=9/10=0.90$
- Toy Story و Matrix: $(3+3)/10=0.60$
- Toy Story و Terminator: $3/10=0.30$
- Toy Story و Titanic: 0

برخلاف Jaccard ژانر، مقدارها **درجه‌بندی** دارند: Cars و Nemo هر دو «خیلی نزدیک» ولی مساوی نیستند، و Matrix «تا حدی مرتبط» است نه صفر.

مثال ۴ (همسایه‌های واقعی روی MovieLens). برای Toy Story: با ژانر one-hot، پنج همسایه‌ی اول همه شباهت ۱٫۰۰ دارند و فیلم‌هایی دلبخواهی‌اند (tie). با TF-IDF: Toy Story 2 (۰٫۶۹)، Bug's Life، Toy Story 3. با embedding خود استاد: Toy Story 2، Bug's Life، Toy Story 3، Lego Movie — چون «pixar tag» و عنوان مشترک را می‌بینند. برای Matrix، embedding استاد: Matrix Revolutions (۰٫۶۶)، Matrix Reloaded (۰٫۶۱).

	One-hot ژانر	TF-IDF	Embedding
مترادف را می‌فهمد	نه	نه	**بله**
توضیح‌پذیر	**بله**	**بله** (کلمه‌ی مشترک)	نه
هزینه	تقریباً صفر	ارزان	مدل عصبی (GPU کمک می‌کند)
کاربرد معمول	فیلتر، baseline	جست‌وجو، متن زیاد	recommendation و جست‌وجوی معنایی مدرن

۴.۶ Top-N با پروفایل کاربر

استاد Top-N را مثل فصل ۲ دو جور می‌سازد: **Predict & Sort** و **kNN-map** (با heap؛ بخش ۲.۸). روش سومی هست که ساده‌تر است و در عمل بیشتر استفاده می‌شود: **برای کاربر یک بردار بساز («پروفایل کاربر») و با بردار همه‌ی کالاها مقایسه کن.**

1. **پروفایل** = میانگین بردار کالاهایی که کاربر **دوست داشته** (مثلاً امتیاز ۴ به بالا).
2. **score** هر کاندید = cosine(پروفایل، بردار کالا).
3. بالاترین score ها از بین کالاهای ندیده را پیشنهاد بده.

مثال ۱ (Ana با بردارهای اسباب‌بازی). Ana فیلم‌های Toy Story (۵) و Nemo (۴) را دوست داشت؛ Matrix را نه، پس در پروفایل نیست.

$$\text{profile} = \frac{[3,1,0]+[3,0,1]}{2} = [3,\ 0.5,\ 0.5] \qquad \|\text{profile}\| = \sqrt{9+0.25+0.25} = 3.08$$

کاندید	بردار	ضرب داخلی	طول	cosine
Cars	$[3,2,0]$	$9+1+0=10$	3.61	$10/(3.08\times 3.61) = 0.90$
Terminator	$[0,3,1]$	$0+1.5+0.5=2$	3.16	$2/(3.08\times 3.16) = 0.21$
Titanic	$[0,0,3]$	$0+0+1.5=1.5$	3.00	$1.5/(3.08\times 3.00) = 0.16$

لیست Ana: **Cars، Terminator، Titanic**.

مثال ۲. کاربری کالاهایی با بردار $[1,0]$ و $[0.6,0.8]$ را دوست داشته. کاندیدها: $P=[1,1]$ و $Q=[0,1]$.

پروفایل $=([1,0]+[0.6,0.8])/2 = [0.8,\ 0.4]$؛ طول $\sqrt{0.64+0.16} = 0.894$.

$\cos(\text{profile},P) = \frac{0.8+0.4}{0.894\times 1.414} = \frac{1.2}{1.265} = 0.949$؛ $\cos(\text{profile},Q) = \frac{0.4}{0.894\times 1} = 0.447$ ← اول P.

چرا صنعت روش پروفایل را ترجیح می‌دهد؟ یک بردار برای هر کاربر؛ امتیاز دادن به همه‌ی کالاها یک ضرب ماتریس در بردار است ($O(nd)$)، و با یک **ANN index** (FAISS)، یا پایگاه‌داده‌های برداری مثل pgvector، Milvus، Qdrant، Pinecone فقط

بخش کوچکی از کاتالوگ لمس می‌شود. کالای جدید هم کار خاصی لازم ندارد: به محض این‌که بردار داشت، امتیاز می‌گیرد. این دقیقاً الگوی **two-tower** سیستم‌های بزرگ است: یک بردار برای کاربر، یک بردار برای کالا، score = ضرب داخلی.

۴.۷ ارزیابی منصفانه: split زمانی، Hit Ratio و NDCG

split تصادفی در برابر split زمانی. تاریخچه‌ی Ana به ترتیب زمان: ژانویه (۵) Toy Story، فوریه (۴) Nemo، مارس (۱) Matrix، آوریل (۵) Up.

- **split تصادفی** ممکن است آوریل را در train و ژانویه را در test بگذارد: مدل از **آینده** برای پیش‌بینی **گذشته** یاد می‌گیرد. در محیط واقعی این هرگز اتفاق نمی‌افتد؛ همیشه «بعدی» را پیش‌بینی می‌کنیم. پس split تصادفی عدد خوش‌بینانه می‌دهد.
- **leave-one-out زمانی** آخرین تعامل مثبت هر کاربر (اینجا Up در آوریل) را کنار می‌گذارد و روی همه‌ی قبلی‌ها train می‌کند. **ارزیابی نهایی استاد دقیقاً همین را انجام می‌دهد** (`sort_values('timestamp')` و بعد `groupby('userId').tail(1)`). این روش صنعتی است و گفتنش در امتحان امتیاز دارد.

Hit Ratio و NDCG با یک کالای test برای هر کاربر (تمرین ۸ استاد). اگر کالای test در رتبه‌ی p (۱ = بالا) داخل top-N باشد:

$$\text{hit} = 1, \qquad \text{NDCG} = \frac{1}{\log_2(p+1)}$$

چرا فرمول اینقدر ساده شد؟ با دقیقاً یک کالای مرتبط، لیست ایده‌آل آن را در رتبه‌ی ۱ می‌گذارد، پس $\text{IDCG} = 1/\log_2 2 = 1$ و NDCG همان DCG است. با چند کالای مرتبط (فصل ۳) باید بر IDCG واقعی تقسیم کنی.

مثال ۱ (تمرین ۸ استاد).

رتبه	$\log_2(p+1)$	NDCG
1	$\log_2 2 = 1$	1.000
3	$\log_2 4 = 2$	0.500
7	$\log_2 8 = 3$	0.333
خارج از top-N	–	0

مثال ۲ (HR یکسان، NDCG متفاوت؛ تمرین ۷ استاد). دو کاربر test. روش A کالای درست را در رتبه‌های ۱ و ۳ می‌گذارد؛ روش B در رتبه‌های ۹ و ۱۰. هر دو HR@10 $= 2/2 = 1.0$ دارند.

- NDCG روش A $= (1 + 0.5)/2 = 0.75$
- NDCG روش B $= (1/\log_2 10 + 1/\log_2 11)/2 = (0.301 + 0.289)/2 = 0.295$

روش A کالای درست را بالاتر می‌گذارد، همان‌جا که کاربر واقعاً نگاه می‌کند.

۴.۸ Content در برابر CF، و روش‌های ترکیبی (Hybrid)

	Collaborative filtering	Content-based
یاد می‌گیرد از	رفتار کاربران دیگر	توصیف خود کالاها
کالای جدید	**نمی‌تواند**	**می‌تواند**
کاربر جدید	نمی‌تواند	نمی‌تواند
مفهوم کیفیت	**دارد**	ندارد
پیشنهاد غافلگیرکننده	**بله** (ارتباط‌های پنهان)	به‌ندرت (بیشتر از همان)
توضیح‌پذیر	تا حدی	**بله**
لازم دارد	داده‌ی تعامل زیاد	ویژگی خوب کالا

چرا item-based CF وقتی داده زیاد است از content بهتر است؟ (بخش استاد) (۱) **رفتار در برابر متن:** رفتار واقعی سلیقه‌های ناگفته را می‌گیرد («خرد جمعی»)؛ (۲) **کیفیت:** content نمی‌داند فیلم خوب است یا نه؛ (۳) **ویژگی‌های کم:** embedding استاد فقط عنوان، ژانر و tag های کم را دارد (فقط حدود ۱۶٪ فیلم‌ها اصلاً tag دارند)؛ (۴) **serendipity:** CF ارتباط‌های غیرواضح بین ژانرهای مختلف را پیدا می‌کند.

آیا content وقتی امتیاز کافی داریم هنوز مفید است؟ (بخش استاد) **قطعاً:** (۱) غلبه بر popularity bias و رساندن کالاهای long-tail؛ (۲) توضیح‌پذیری («چون تم cyberpunk و dystopia را با Matrix مشترک دارد»)؛ (۳) پاسخ به حال‌وهوای خاص کاربر؛ (۴) **سیستم‌های hybrid**.

انواع hybrid:

1. **Weighted:** $\text{score} = \alpha \cdot \text{CF} + (1-\alpha) \cdot \text{content}$.
2. **Switching:** وقتی کالا تعامل کافی دارد CF، وگرنه content (کالای جدید).
3. **Feature combination:** یک مدل که هم داده‌ی تعامل و هم ویژگی کالا را می‌گیرد (مثل LightFM یا شبکه‌های two-tower).
4. **Cascade / دو مرحله‌ای:** یک روش چند صد کاندید می‌سازد و روش دیگر دوباره مرتبشان می‌کند. استاد همین را توصیف می‌کند: CF هزار کاندید خوب می‌سازد و یک مدل محتوایی ۱۰ تای نهایی را شخصی‌سازی می‌کند.

مثال ۱ (weighted + switching، $\alpha = 0.6$، برای Ana).

کاندید	score CF	score محتوا	score نهایی
Up (قدیمی)	0.80	0.90	$0.6 \times 0.80 + 0.4 \times 0.90 = 0.84$
Terminator (قدیمی)	0.10	0.21	$0.6 \times 0.10 + 0.4 \times 0.21 = 0.14$
Cars (جدید)	— (داده ندارد)	0.90	switch به محتوا ← **0.90**

در عمل اول دو score روی یک مقیاس آورده می‌شوند (مثلاً تقسیم بر بیشینه‌ی هر کاربر)، وگرنه یکی فقط به خاطر اعداد بزرگ‌تر غالب می‌شود.

مثال ۲. کالای قدیمی با 0.6 CF و محتوا 0.9؛ کالای جدید فقط محتوا 0.8؛ $\alpha = 0.5$ با switching: قدیمی $= 0.5 \times 0.6 + 0.5 \times 0.9 = 0.75$؛ جدید $= 0.8$ ← کالای جدید بالاتر می‌نشیند.

آزمایش روی MovieLens با split زمانی استاد (HR@20):

- **کاربران با تاریخچه‌ی زیاد:** item-kNN CF ۰٫۲۱؛ popularity ۰٫۱۴؛ پروفایل محتوایی تنها فقط ۰٫۰۱۵ تا ۰٫۰۴ (زیر popularity، چون مفهوم کیفیت ندارد)؛ محتوا + محبوبیت ۰٫۰۹ تا ۰٫۱۳؛ hybrid حدود ۰٫۲۲ (کمی بهتر از CF).
- **فیلم‌های جدید (cold start)** — ۱۰٪ فیلم‌ها را «جدید» فرض کردیم و همه‌ی امتیازهایشان را از train حذف کردیم (HR@10 در ردیف «تازه‌ها»): random ۰٫۰۲۷؛ **CF ۰٫۰۲۷ (هم‌اندازه‌ی random)**؛ محتوا ۰٫۰۵ تا ۰٫۱۶ (تا حدود ۶ برابر random).

این جدول دوم دلیل وجود content-based است.

۴.۹ الگوریتم‌ها و پیچیدگی

```
CONTENT_PREDICT(u, i): num <- 0 ; den <- 0 for each item j rated by u: s <- sim(content[i], content[j])
# Jaccard on sets / cosine on vectors num <- num + s * r[u][j] ; den <- den + s return num / den
(fallback if den = 0) CONTENT_TOPN_PROFILE(u, N): profile <- average of vec[j] over items j that u
liked for each item i not seen by u: score[i] <- cosine(profile, vec[i]) return the N items with the
highest score
```

n کالا، d طول بردار (۲۰ ژانر، چند هزار کلمه‌ی TF-IDF، ۳۸۴ عدد embedding)، $|I_u|$ کالاهای کاربر.

عملیات	هزینه	توضیح
یک similarity	$O(g)$ برای مجموعه، $O(d)$ برای بردار	
`CONTENT_PREDICT`	$O(\|I_u\| \cdot d)$	
Top-N Predict & Sort	$O(n \cdot \|I_u\| \cdot d)$ برای هر کاربر	برای online خیلی کند
ساخت kNN-map	زمان $O(n^2 d)$، حافظه $O(nk)$	offline؛ بردار کالا عوض نمی‌شود، پس یک بار
Top-N با kNN-map	$O(\|I_u\| \cdot k)$	
Top-N با پروفایل	$O(\|I_u\| d + nd)$، یا حدود $O(\log n)$ با ANN	روش مقیاس بزرگ

یک مزیت نسبت به CF: شباهت محتوایی فقط به کالاها بستگی دارد نه رفتار کاربران؛ با امتیاز دادن کاربران عوض نمی‌شود، پس kNN-map یک بار ساخته و فقط با آمدن کالای جدید گسترش داده می‌شود.

۴.۱۰ کد استاد (`content_based.ipynb`) و ایرادهایش

داده و جدول‌ها: `rating_map` (این بار با dictionary comprehension سریع)، و `movie_genres[movieId]` ← مجموعه‌ی ژانرها با `set(g.split('|'))` تا بشود از `&` و `|` (اشتراک و اجتماع) استفاده کرد.

`item_sim(i, j, sim_type)`: برای `'genre'` Jaccard روی مجموعه‌ها؛ برای `'embedding'` cosine با `np.dot(e_i, e_j) / (norm(e_i)*norm(e_j) + 1e-9)` (عدد 1e-9 از تقسیم بر صفر جلوگیری می‌کند).

`content_based_rating_prediction(u, i)`: میانگین وزن‌دار بخش ۴.۴ روی **همه‌ی** فیلم‌های کاربر.

`build_knn_map`: این بار برخلاف هفته‌ی ۲ همه‌ی شباهت‌ها را با عملیات ماتریسی یک‌جا حساب می‌کند: ژانرها با `MultiLabelBinarizer` به ماتریس صفر و یک G (فیلم × ژانر) تبدیل می‌شوند؛ **اشتراک** $= GG^\top$ (ضرب داخلی دو سطر صفر و یک، ژانرهای مشترک را می‌شمارد)؛ اجتماع $= |A| + |B| -$ اشتراک؛ Jaccard = اشتراک / اجتماع. برای embedding، `cosine_similarity(emb_matrix)`. سپس `np.argpartition(sims, -(K+1))` $K+1$ بزرگ‌ترین را **بدون مرتب کردن کل سطر** پیدا می‌کند ($O(n)$ به جای $O(n \log n)$)؛ +۱ چون خود فیلم همیشه شبیه‌ترین است و کنار گذاشته می‌شود.

ارزیابی نهایی: split زمانی leave-one-out، تاریخچه‌ی کاربر فقط از امتیازهای ۴+، Hit Ratio@20 و NDCG@20. خروجی استاد: ژانر NDCG 0.0067، HR 0.0019؛ embedding NDCG 0.040، HR 0.016.

ایرادهای کد استاد:

1. **Leakage جدی در ارزیابی rating prediction.** استاد `ratings` را به `X_train` و `X_test` تقسیم می‌کند ولی `content_based_rating_prediction(u, i)` از `ratings` و `rating_map` **سراسری** (کل داده، شامل امتیاز test) می‌خواند. برای جفت test (u, i) خود: (۱) `movies_rated` جزو i است؛ (۲) `item_sim(i, i)` $= 1$، بیشترین وزن ممکن؛ (۳) پس امتیاز واقعی r_{ui} با بیشترین وزن وارد میانگین می‌شود. **جواب را به مدل نشان داده‌ایم و بعد خواسته‌ایم پیش‌بینی‌اش کند.** وقتی همین ارزیابی را بدون leakage (فقط با داده‌ی train) تکرار کردیم، MAE ژانر از حدود ۰٫۷۰ به ۰٫۷۲ بدتر شد؛ یعنی عدد استاد خوش‌بینانه بود.
2. **ناسازگاری متن و خروجی:** متن استاد می‌گوید embedding به «Hit Ratio ~0.26 در برابر ~0.12» برای ژانر رسید؛ خروجی سلول خودش ۰٫۰۴۰ در برابر ۰٫۰۰۷ است (احتمالاً متن برای نسخه‌ی قبلی کد است). در امتحان به استدلال تکیه کن، نه به آن اعداد.
3. **Tie در ژانر:** `knn_map[3]` سی فیلم با شباهت ۱٫۰ است که دلبخواهی از بین صدها tie انتخاب شده‌اند؛ همین HR نزدیک صفر kNN-map ژانر را توضیح می‌دهد.
4. **حافظه:** ماتریس کامل 9742×9742 چند بار ساخته می‌شود (اشتراک، اجتماع، نتیجه)، هر کدام حدود ۷۶۰ مگابایت؛ برای یک میلیون کالا حدود ۸ ترابایت. در عمل chunk به chunk و فقط top-k.
5. **`top_N_pred_sort`:** ۹,۵۰۰ پیش‌بینی برای هر کاربر؛ بالای لیستش چند فیلم با پیش‌بینی ۵٫۰ است: کاندیدی که فقط با فیلم‌هایی که کاربر ۵ داده ژانر مشترک دارد (اغلب فقط یکی)، همان امتیاز را عیناً می‌گیرد — مصنوعی، نه پیشنهاد عالی.
6. **همان دو باگ هفته‌ی ۲:** `groupby` بدون sort در `add_sims_and_sort`، و حذف نشدن فیلم‌های دیده‌شده.
7. `ratings[ratings.userId == u]` در هر پیش‌بینی ۱۰۰ هزار ردیف را اسکن می‌کند؛ `item_sim` طول بردارها را هر بار دوباره حساب می‌کند (بهتر: یک بار نرمال کن، بعد cosine = ضرب داخلی).

۴.۱۱ سوال‌های امتحان — فصل ۴

تمرین‌های خود استاد:

تمرین ۱. امتیاز User 2 به Toy Story با embedding ← فراخوانی مستقیم تابع `content_based_rating_prediction(2, 1, sim_type='embedding')`؛ خروجی استاد ۳٫۹۲.

تمرین ۲ (kNN). فقط k فیلمِ شبیه‌ترِ امتیازداده: شباهت با همه‌ی فیلم‌های امتیازداده را حساب کن، k بزرگ‌ترین را نگه دار (heap اندازه‌ی k با $O(|I_u| \log k)$؛ یا sort نزولی و برش `[:k]` مثل حل استاد)، و فقط روی آن‌ها میانگین وزن‌دار بگیر. خروجی استاد با $k = 5$: ۳٫۸۶.

تمرین ۳. ۵ فیلم شبیه Matrix (2571) بدون خودش ← cosine embedding با همه، مرتب کردن نزولی، رد کردن خود فیلم. جواب استاد: Peaceful Warrior، Iron Soldier، Art of War، Matrix Reloaded، Matrix Revolutions.

تمرین ۴ (cold start). فیلم جدید با صفر امتیاز را کدام می‌تواند پیشنهاد دهد؟ CF **Content-based.** شباهت (یا بردار) کالا را از تعامل‌ها می‌سازد؛ کالای بدون تعامل شباهتی با هیچ‌چیز ندارد. content فقط توصیف خود کالا (عنوان، ژانر، tag) را لازم دارد که از لحظه‌ی اضافه شدن موجود است. ::: {dir="ltr"} *English:* Content-based. CF derives item similarity from interactions; with no interactions the new item has no similarity to anything. Content-based needs only the item's metadata, available as soon as it is added. :::

تمرین ۵. چرا embedding می‌تواند HR خیلی بهتری از ژانر بگیرد در حالی که MAE مشابه دارند؟ MAE فقط نزدیکی امتیاز را روی کالاهایی که کاربر امتیاز داده می‌سنجد؛ امتیازها دور میانگین کاربر جمع‌اند و هر دو similarity فقط **وزن** یک میانگین‌اند، پس هر دو نزدیک میانگین کاربر می‌افتند. HR باید کالای درست را بالاتر از هزاران کالای دیگر بنشاند؛ ژانر خشن است و tie های زیاد می‌سازد و کالای درست بین tie ها گم می‌شود؛ embedding شباهت درجه‌بندی‌شده و ظریف‌تر می‌دهد و آن را بالا می‌آورد. ::: {dir="ltr"} *English:* MAE only checks closeness of ratings on rated items; both similarities act as weights and predictions collapse near the user's mean. Hit Ratio requires ranking the right item above thousands; coarse genres create massive ties, embeddings give graded similarities that separate items. :::

تمرین ۶. دو ویژگی اضافه برای embedding: **خلاصه‌ی داستان** و **کارگردان و بازیگران اصلی** (یا: کلیدواژه‌ها، نقدها، سال/دوره، زبان، استودیو).

تمرین ۷. HR@100 برابر ولی NDCG@100 روش A خیلی بالاتر: هر دو کالای درست را به یک اندازه داخل top-100 می‌آورند، ولی A آن را **بالاتر** می‌گذارد؛ HR دودویی است (رتبه‌ی ۱ و ۱۰۰ یکی‌اند) ولی NDCG با $1/\log_2(\text{rank}+1)$ رتبه‌های بالا را بیشتر پاداش می‌دهد. (مثال ۲ بخش ۴.۷.)

تمرین ۸. رتبه‌ی ۳ ← ۰٫۵؛ رتبه‌ی ۱ ← ۱؛ رتبه‌ی ۷ ← ۰٫۳۳۳ (بخش ۴.۷). فقط برای **یک** کالای مرتبط. :::

سوال‌های متن نوت‌بوک:

- **«Can you suggest a solution?»** (کارایی) — همه‌ی کالاها را پیش‌بینی نکن: شباهت‌های محتوایی را offline حساب کن و فقط top-k هر کالا را نگه دار (چون محتوا عوض نمی‌شود، یک بار)؛ موقع درخواست یا همسایه‌های کالاهای کاربر را جمع بزن (kNN-map) یا پروفایل کاربر را به یک ANN index بده و فقط چند صد کاندید را رتبه‌بندی کن.
- **«Why don't embeddings always improve rating prediction?»** — (۱) rating prediction رگرسیون است و تحت تأثیر bias کاربر و کالا؛ similarity فقط وزن یک میانگین است، پس پیش‌بینی به میانگین کاربر میل می‌کند؛ embedding در **رتبه‌بندی** خیلی بهتر است. (۲) sparsity ویژگی: tag ها کم‌اند، پس embedding برای بیشتر فیلم‌ها فقط عنوان و ژانر را کد می‌کند.
- **«Why does item-based CF outperform content-based?»** و **«Is content still useful?»** — بخش ۴.۸.
- **«What should be the value of X?»** (آستانه‌ی امتیاز برای تاریخچه‌ی مثبت کاربر) — آستانه باید فقط چیزهایی را که کاربر واقعاً دوست داشته نگه دارد، چون پروفایل/همسایه‌ها از آن‌ها ساخته می‌شوند؛ امتیاز ۱ و ۲ نباید «شبیه این را بیشتر بده» حساب شود. با میانگین حدود ۳٫۵ در MovieLens، ۴ انتخاب معقولی است (استاد هم ۴ گذاشته)؛ آستانه‌ی دقیق با validation تنظیم می‌شود. یک گزینه‌ی بهتر: آستانه‌ی **نسبی** (بالاتر از میانگین خود کاربر).

شکل‌های محتمل دیگر:

- **content-based را تعریف کن و با CF مقایسه کن.** ← بخش ۴.۱ و جدول ۴.۸.
- **محدودیت‌ها را فهرست کن.** ← کیفیت، over-specialization، وابستگی به ویژگی، cold start کاربر، tie ها.
- **انواع hybrid را نام ببر و توضیح بده.** ← cascade، feature combination، switching، weighted.
- **چرا split زمانی از تصادفی بهتر است؟** ← در عمل همیشه آینده را از گذشته پیش‌بینی می‌کنیم؛ split تصادفی به مدل اجازه‌ی دیدن آینده را می‌دهد و نتیجه را خوش‌بینانه می‌کند.
- **چرا kNN-map ژانری در Top-N بد است؟** ← tie های عظیم با شباهت ۱٫۰؛ همسایه‌ها دلبخواهی‌اند؛ سیگنال کیفیت نیست.
- **ایراد ارزیابی rating استاد چیست؟** ← leakage (بخش ۴.۱۰).
- **چرا برای ساخت kNN-map از heap استفاده می‌شود؟ پیچیدگی؟** ← $O(c \log k)$ زمان و $O(k)$ حافظه در برابر $O(c \log c)$ و $O(c)$.
- **چطور cosine را برای تعداد زیادی کالا ارزان کنیم؟** ← همه‌ی بردارها را یک بار نرمال کن؛ cosine = ضرب داخلی؛ همه‌ی شباهت‌ها = یک ضرب ماتریس؛ serving با ANN.
- **محاسبه‌ی دستی:** Jaccard ژانر و پیش‌بینی (مثال‌های ۴.۴)، TF-IDF (مثال‌های ۴.۵)، پروفایل (مثال‌های ۴.۶)، heap (بخش ۲.۸)، hybrid (۴.۸)، NDCG (۴.۷).

فصل ۵ — جمع‌بندی، فرمول‌نامه و تمرین‌های مرور

۵.۱ کل درس در یک جدول

	Baselines	Item-kNN CF	MF + BPR	LightGCN	Content-based
از چه یاد می‌گیرد	شمارش و میانگین	رفتار کاربران دیگر	رفتار کاربران دیگر	رفتار + ساختار گراف	توصیف خود کالا
شخصی‌سازی	ندارد	دارد	دارد	دارد	دارد
کالای جدید	نه	نه	نه	نه	**بله**
کاربر جدید	بله (popularity)	نه	نه	نه	نه
مفهوم کیفیت	بله	بله	بله	بله	**نه**
Training	ندارد	ندارد (precompute)	SGD / ALS	SGD	ندارد (یا مدل embedding)
ذخیره	$O(n)$	$O(nk)$	$(m+n)d$	$(m+n)d$ + گراف	$O(nd)$
هزینه‌ی offline	$O(\|R\|)$	$O(n^2\bar{u})$	$O(\text{epochs}\cdot\|R\|d)$	$O(\text{steps}\cdot K\|E\|d)$	$O(n^2d)$ برای kNN-map
توضیح‌پذیری	بالا	بالا	کم	کم	بالا
نقش در صنعت	fallback، feature	موتور اصلی با داده‌ی کافی	embedding و بازیابی	وقتی گراف سیگنال دارد	cold start، long tail، re-ranking

داستان در یک پاراگراف. baseline ها به همه یک جواب می‌دهند و نمی‌فهمند Ana اکشن دوست ندارد. item-kNN با similarity درست (adjusted cosine) این را از رفتار دیگران می‌فهمد، ولی $O(n^2)$ است و با sparsity ضعیف می‌شود. MF به هر کاربر و کالا یک بردار کوتاه می‌دهد و با BPR از داده‌ی implicit یاد می‌گیرد؛ LightGCN ایده‌ی همسایگی را داخل این بردارها می‌برد. هیچ‌کدام کالای جدید را نمی‌بینند؛ content-based با استفاده از توصیف کالا این را حل می‌کند ولی کیفیت را نمی‌فهمد. در صنعت این‌ها ترکیب (hybrid) می‌شوند.

۵.۲ فرمول‌نامه

Baselines و خطا

$$\mu = \frac{1}{|R|}\sum r_{ui} \qquad \hat{r}_{ui} = \mu + b_u + b_i \qquad b_i = \frac{\sum_{u\in U_i}(r_{ui}-\mu)}{\lambda + |U_i|} \qquad \text{damped} = \frac{\sum r + \lambda\mu}{n+\lambda}$$

$$\text{MAE} = \frac{1}{|T|}\sum|r-\hat{r}| \qquad \text{RMSE} = \sqrt{\frac{1}{|T|}\sum(r-\hat{r})^2} \qquad \text{HR@}K = \frac{\#\text{hits}}{|T|} \qquad \text{HR@}K_{\text{random}} = \frac{K}{\#\text{candidates}}$$

Similarity و پیش‌بینی

$$J(A,B) = \frac{|A\cap B|}{|A\cup B|}, \quad |A\cup B| = |A| + |B| - |A\cap B| \qquad \cos(\mathbf{a},\mathbf{b}) = \frac{\mathbf{a}\cdot\mathbf{b}}{\|\mathbf{a}\|\|\mathbf{b}\|}$$

$$\text{adjusted cosine: cosine on } (r_{ui}-\bar{r}_u) \qquad \hat{r}_{ui} = \frac{\sum_j s_{ij} r_{uj}}{\sum_j |s_{ij}|} \qquad \hat{r}_{ui} = \bar{r}_u + \frac{\sum_j s_{ij}(r_{uj}-\bar{r}_u)}{\sum_j |s_{ij}|}$$

$$\text{tf-idf}(w,d) = \text{tf}(w,d) \cdot \log \frac{N}{\text{df}(w)} \qquad \text{profile}_u = \text{mean of liked item vectors}, \;\; \text{score} = \cos(\text{profile}, \mathbf{v}_i)$$

معیارهای رتبه‌بندی

$$\text{P@}K = \frac{\text{hits}}{K} \qquad \text{R@}K = \frac{\text{hits}}{|\text{relevant}|} \qquad \text{NDCG} = \frac{\sum_{\text{hits}} 1/\log_2(p+1)}{\sum_{p=1}^{\min(K,|\text{rel}|)} 1/\log_2(p+1)} \qquad \text{single hit: } \frac{1}{\log_2(p+1)}$$

MF، BPR، LightGCN

$$\hat{x}_{ui} = \mathbf{p}_u^\top \mathbf{q}_i \qquad \text{loss} = -\ln \sigma(\hat{x}_{ui} - \hat{x}_{uj}) \qquad g = 1 - \sigma(\hat{x}_{ui} - \hat{x}_{uj})$$

$$\mathbf{p}_u \mathrel{+}= \eta g(\mathbf{q}_i - \mathbf{q}_j), \quad \mathbf{q}_i \mathrel{+}= \eta g \mathbf{p}_u, \quad \mathbf{q}_j \mathrel{-}= \eta g \mathbf{p}_u$$

$$A = \begin{bmatrix} 0 & R \\ R^\top & 0 \end{bmatrix} \qquad \hat{A} = D^{-1/2} A D^{-1/2} \quad (\text{edge weight } 1/\sqrt{\deg_i \deg_j}) \qquad E^{(k+1)} = \hat{A} E^{(k)} \qquad E = \frac{1}{K+1} \sum_{k=0}^{K} E^{(k)}$$

۵.۳ عددهای مرجع

مقدار	عدد
$1/\log_2(p+1)$ برای رتبه‌ی ۱، ۲، ۳، ۴، ۵، ۷	۱، ۰٫۶۳۱، ۰٫۵، ۰٫۴۳۱، ۰٫۳۸۷، ۰٫۳۳۳
$\sigma(0)$ و $-\ln 0.5$ در شروع BPR (train)	۰٫۵ و ۰٫۶۹۳
روش تصادفی در پروتکل ۹۹+۱ HR@10	۰٫۱۰
MovieLens small دیتاست sparsity	حدود ۹۸٫۳٪
تعداد جفت برای n کالا	$n(n-1)/2$
$1/\sqrt{2}$، $1/\sqrt{3}$، $1/\sqrt{6}$	۰٫۷۰۷، ۰٫۵۷۷، ۰٫۴۰۸

۵.۴ اشتباه‌های رایج در امتحان

1. در cosine دستی (روش استاد) بردارها فقط روی **کاربران مشترک** گرفته می‌شوند؛ ولی در Jaccard اجتماع شامل **همه‌ی** کاربرانِ هر دو کالاست.
2. مخرج پیش‌بینی **جمع شباهت‌ها**ست، نه تعداد همسایه‌ها.
3. مخرج Recall تعداد کالاهای **مرتبط** است؛ مخرج Precision خود **K**.
4. IDCG را با $\min(K, |\text{relevant}|)$ جمله حساب کن؛ میان‌بر $1/\log_2(p+1)$ فقط برای **یک** کالای مرتبط است.
5. MAE کمتر بهتر است؛ Recall، Precision، HR و NDCG بیشتر بهتر.
6. «Jaccard مقدار امتیاز را نادیده می‌گیرد» به معنی «Jaccard بدتر است» **نیست**؛ در نوت‌بوک استاد بهتر شد و باید بتوانی دلیلش را بگویی.
7. content-based مسئله‌ی cold start **کالا** را حل می‌کند، نه **کاربر**.
8. density را با sparsity اشتباه نگیر.
9. HR های اندازه‌گیری‌شده با پروتکل‌های متفاوت (۹۹+۱ در برابر کل کاتالوگ) قابل مقایسه نیستند.
10. به ارزیابی‌ای که داده‌ی test به مدل می‌رسد (leakage) اعتماد نکن.

۵.۵ تمرین‌های مرور (با جواب)

سعی کن اول خودت حل کنی، بعد جواب را ببین.

تمرین ۱ (baseline). امتیازهای train: کاربر X: $4, 5$؛ کاربر Y: $2, 3, 1$. فیلم F امتیازهای 5 (از X) و 1 (از Y) را دارد. (الف) μ؛ (ب) $\bar{r}_X$ و $\bar{r}_Y$؛ (ج) پیش‌بینی bias baseline ساده برای F (Y → F).

جواب. (الف) $\mu = (4+5+2+3+1)/5 = 15/5 = 3.0$. (ب) $\bar{r}_X = 4.5$، $\bar{r}_Y = 2.0$. (ج) $\bar{r}_F = (5+1)/2 = 3.0$ ← $b_Y = 2.0 - 3.0 = -1.0$، $b_F = 3.0 - 3.0 = 0$ ← $\hat{r} = 3.0 - 1.0 + 0 = 2.0$.

تمرین ۲ (MAE/RMSE). واقعی $[5, 3, 1]$، پیش‌بینی $[4, 3, 3]$.

جواب. خطاها $1, 0, 2$ ← MAE $= 3/3 = 1.0$؛ مربع‌ها $1, 0, 4$ ← RMSE $= \sqrt{5/3} = 1.29$.

تمرین ۳ (damped average). $\mu = 3.5$، $\lambda = 5$. فیلم P: ۲ امتیاز با میانگین ۵؛ فیلم Q: ۵۰ امتیاز با میانگین ۴٫۴. کدام بالاتر است؟

جواب. P: $(10 + 17.5)/7 = 3.93$؛ Q ← $(220 + 17.5)/55 = 4.32$: Q بالاتر، با این‌که میانگین ساده‌ی P بیشتر بود.

تمرین ۴ (similarity). دو کالا روی کاربران مشترک: $\vec{a} = [5, 1]$، $\vec{b} = [1, 5]$. میانگین کاربر اول ۳ و کاربر دوم ۳ است. cosine خام و adjusted cosine را حساب کن.

جواب. خام: $(5+5)/(\sqrt{26}\sqrt{26}) = 10/26 = 0.38$ — مثبت با این‌که دو کاربر کاملاً مخالف‌اند. centered: $\vec{a}' = [2, -2]$، $\vec{b}' = [-2, 2]$ ← $(-4-4)/(\sqrt{8}\sqrt{8}) = -8/8 = -1.0$ ← کاملاً ضد هم.

تمرین ۵ (پیش‌بینی kNN). کاربر به A، B، C به ترتیب $4, 2, 5$ داده. شباهت با هدف: $0.6,\ 0.1,\ 0.9$. پیش‌بینی با $k = 3$ و $k = 2$.

جواب. $k = 3$: $(3.6 + 0.2 + 3.0)/1.6 = 6.8/1.6 = 4.25$. $k = 2$ (A و C): $(3.6 + 3.0)/1.5 = 6.6/1.5 = 4.4$.

تمرین ۶ (kNN-map). کاربر X و Y را دیده؛ $\text{knn}[X] = [(0.7, P), (0.2, Q)]$، $\text{knn}[Y] = [(0.5, Q), (0.4, R)]$. top-2 با جمع و با ماکسیمم.

جواب. جمع: $P = 0.7$، $Q = 0.7$، $R = 0.4$ ← P و Q (مساوی). ماکسیمم: $P = 0.7$، $Q = 0.5$، $R = 0.4$ ← Q، P.

تمرین ۷ (heap). top-2 از $0.6,\ 0.3,\ 0.8,\ 0.1,\ 0.4$ با min-heap.

جواب. $[0.4]$ ← $[0.1, 0.4]$ ← 0.8 بیشتر از 0.1، جایگزین ← $[0.4, 0.8]$ ← 0.3 کمتر از 0.4، نادیده ← 0.6 بیشتر از 0.4، جایگزین ← $[0.6, 0.8]$. نتیجه: ۰٫۸ و ۰٫۶.

تمرین ۸ (NDCG). top-4: $[x, y, z, w]$؛ مرتبط‌ها $\{y, w\}$.

جواب. P@4 $= 2/4 = 0.5$؛ R@4 $= 2/2 = 1$؛ DCG $= 0.631 + 0.431 = 1.062$؛ IDCG $= 1 + 0.631 = 1.631$؛ NDCG $= 0.651$.

تمرین ۹ (MF و BPR). $\mathbf{p}_u = [1, 2]$، $\mathbf{q}_i = [2, 1]$، $\mathbf{q}_j = [1, 1]$. score ها، gap و loss.

جواب. $\hat{x}_{ui} = 2 + 2 = 4$؛ $\hat{x}_{uj} = 1 + 2 = 3$؛ gap $= 1$؛ $\sigma(1) = 0.731$؛ loss $= -\ln 0.731 = 0.313$. ترتیب درست است، پس loss کوچک و $g = 1 - 0.731 = 0.269$ (اصلاح کوچک).

تمرین ۱۰ (LightGCN). گراف: کاربر u با کالاهای a و b؛ کالای b با کاربر v هم. درجه‌ها و وزن یال‌ها؟

جواب. درجه‌ها: $u = 2$، $v = 1$، $a = 1$، $b = 2$. وزن‌ها: a–u: $1/\sqrt{2 \cdot 1} = 0.707$؛ b–u: $1/\sqrt{2 \cdot 2} = 0.5$؛ b–v: $1/\sqrt{1 \cdot 2} = 0.707$.

تمرین ۱۱ (content-based). کاربر به M1 {Drama, War} نمره‌ی ۵ و به M2 {Comedy} نمره‌ی ۲ داده. پیش‌بینی برای N {Drama, History, War} با Jaccard ژانر.

جواب. $J(N, M1) = 2/3 = 0.667$؛ $J(N, M2) = 0/4 = 0$ ← $\hat{r} = (0.667 \times 5)/0.667 = 5.0$. (فقط از یک فیلم ساخته شده؛ همان ضعف «شواهد کم».)

تمرین ۱۲ (hybrid و cold start). چرا CF روی «ردیف فیلم‌های جدید» نزدیک random است و content بهتر؟ و switching hybrid چه می‌کند؟

جواب. فیلم جدید هیچ تعاملی ندارد، پس CF به همه‌ی فیلم‌های جدید score صفر می‌دهد و ترتیبش عملاً تصادفی است؛ content فقط توصیف فیلم را لازم دارد که از روز اول موجود است. switching hybrid برای کالاهای با تعامل کافی CF و برای کالاهای جدید content را استفاده می‌کند.